DATA-260 — Homework 5

Xu Wang

Repository: https://github.com/xuwang-goldenwater/data260-9275

Section 0 — Personal configuration and domain

Value	This submission	How it is derived
SID4	9275	last four digits of the SJSU Student ID
PORT_BASE	8275	8000 + (9275 mod 900)
PREFIX	s9275	"s" + SID4
SEED	9275	SID4
VERIFY_SEED	269275	260000 + SID4
DOMAIN_ID	3 — grocery supply and recall notices	9275 mod 8

Item	Value
Hardware	MacBook Pro, Apple M1, 8 GB RAM, macOS
Local model	qwen3:8b via Ollama (temperature 0, thinking off), called through src/model_client.py
Database	MySQL 8.0.43, database s9275_rel
Stack	Python 3.12 (conda env data260), FastAPI + SQLAlchemy 2 + PyMySQL, MCP Python SDK 1.x (FastMCP, STDIO), MCP Inspector 2.9; React 18 + Vite + Redux Toolkit + Axios
Commit	f03debd — the code state checked by make verify-hw05 (61 of 61 checks passed)
Tag	hw5 — the commit directly after f03debd. It adds only this report, verification.json and run notes; no application code changes.

Homework 5 extends the same codebase. The HW4 API in code/auth_app/ gains a full second entity (firms) and a relationship query; the React client in frontend/ moves its data layer to Redux Toolkit; the MCP servers, tool layer, tests and agent loop are new files in code/. reports/hw05/ holds evidence only: RUN_LOG.txt, raw/, METRICS.md, TOOL_CONTRACTS.md, AI_USE.md, REFLECTION.md, verification.json and this report.

Part 1 — Second entity, CRUD API and Redux client

The primary entity is a recall notice (table recall_notices). The related entity is the firm that issued it (table firms). One firm has many notices.

1.1 Database

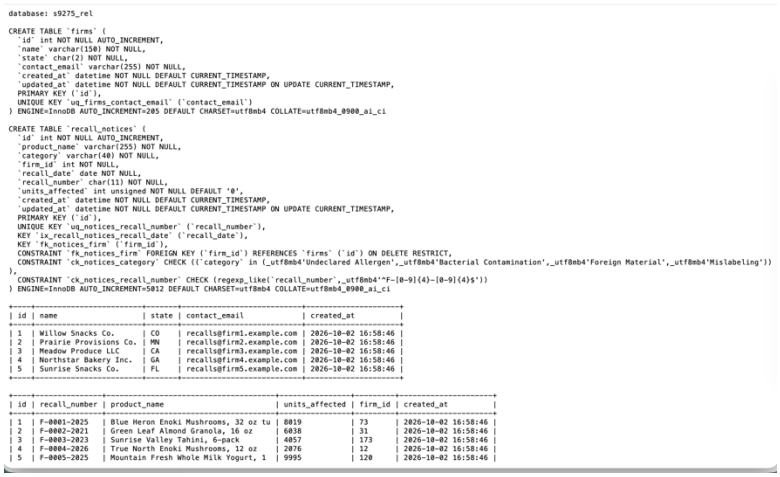
The HW4 tables already held 200 firms and 5,000 notices, so the schema was changed in place by code/sql/hw05_migrate.sql (make migrate-hw05) instead of being rebuilt.

Required field	firms (related entity)	recall_notices (primary entity)
id (primary key)	id INT AUTO_INCREMENT	id INT AUTO_INCREMENT
Primary text field	name VARCHAR(150)	product_name VARCHAR(255)
Secondary text field	state CHAR(2)	category VARCHAR(40), CHECK on 4 values
Unique field	contact_email VARCHAR(255), UNIQUE	recall_number CHAR(11), UNIQUE, CHECK format F-NNNN-YYYY
Numeric field with default	—	units_affected INT UNSIGNED DEFAULT 0
Foreign key	—	firm_id INT NOT NULL → firms.id, ON DELETE RESTRICT
Timestamps	created_at, updated_at	created_at, updated_at

code/sql/hw05_migrate.sql — step 8, the constraints on recall_notices

```
-- 8. Tighten and constrain.
--    UNIQUE  : two notices can never share a recall number
--    CHECK   : the database itself rejects a badly formatted number
--    RESTRICT: deleting a firm that still has notices is refused (error 1451)
ALTER TABLE recall_notices
  MODIFY COLUMN firm_id       INT      NOT NULL,
  MODIFY COLUMN recall_number CHAR(11) NOT NULL,
  ADD CONSTRAINT uq_notices_recall_number UNIQUE (recall_number),
  ADD CONSTRAINT ck_notices_recall_number
      CHECK (recall_number REGEXP '^F-[0-9]{4}-[0-9]{4}$'),
  ADD CONSTRAINT fk_notices_firm
      FOREIGN KEY (firm_id) REFERENCES firms (id) ON DELETE RESTRICT,
  ADD COLUMN created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ADD COLUMN updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
                                 ON UPDATE CURRENT_TIMESTAMP;
```

- Deleting a firm that still has notices is refused by the database itself (ON DELETE RESTRICT, MySQL error 1451). No cascade is used.
- Passwords are not stored in plain text: users.password_hash holds bcrypt hashes (unchanged from HW4).



```
database: s9275_rel

CREATE TABLE `firms` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(150) NOT NULL,
  `state` char(2) NOT NULL,
  `contact_email` varchar(255) NOT NULL,
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uq_firms_contact_email` (`contact_email`)
) ENGINE=InnoDB AUTO_INCREMENT=205 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

CREATE TABLE `recall_notices` (
  `id` int NOT NULL AUTO_INCREMENT,
  `product_name` varchar(255) NOT NULL,
  `category` varchar(40) NOT NULL,
  `firm_id` int NOT NULL,
  `recall_date` date NOT NULL,
  `recall_number` char(11) NOT NULL,
  `units_affected` int unsigned NOT NULL DEFAULT '0',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uq_notices_recall_number` (`recall_number`),
  KEY `ix_recall_notices_recall_date` (`recall_date`),
  KEY `fk_notices_firm` (`firm_id`),
  CONSTRAINT `fk_notices_firm` FOREIGN KEY (`firm_id`) REFERENCES `firms` (`id`) ON DELETE RESTRICT,
  CONSTRAINT `ck_notices_category` CHECK ((`category` in (_utf8mb4'Undeclared Allergen',_utf8mb4'Bacterial Contamination',_utf8mb4'Foreign Material',_utf8mb4'Mislabeling'))
),
  CONSTRAINT `ck_notices_recall_number` CHECK (regexp_like(`recall_number`,_utf8mb4'^F-[0-9]{4}-[0-9]{4}$'))
) ENGINE=InnoDB AUTO_INCREMENT=5012 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

+----+---------------------+-------+-----------------------+---------------------+
| id | name                | state | contact_email         | created_at          |
+----+---------------------+-------+-----------------------+---------------------+
| 1  | Willow Snacks Co.   | CO    | recalls@firm1.example.com | 2026-10-02 16:58:46 |
| 2  | Prairie Provisions Co. | MN | recalls@firm2.example.com | 2026-10-02 16:58:46 |
| 3  | Meadow Produce LLC  | CA    | recalls@firm3.example.com | 2026-10-02 16:58:46 |
| 4  | Northstar Bakery Inc. | GA  | recalls@firm4.example.com | 2026-10-02 16:58:46 |
| 5  | Sunrise Snacks Co.  | FL    | recalls@firm5.example.com | 2026-10-02 16:58:46 |
+----+---------------------+-------+-----------------------+---------------------+

+----+---------------+-----------------------------------+----------------+---------+---------------------+
| id | recall_number | product_name                      | units_affected | firm_id | created_at          |
+----+---------------+-----------------------------------+----------------+---------+---------------------+
| 1  | F-0001-2025   | Blue Heron Enoki Mushrooms, 32 oz tu | 8019         | 73      | 2026-10-02 16:58:46 |
| 2  | F-0002-2021   | Green Leaf Almond Granola, 16 oz  | 6038           | 31      | 2026-10-02 16:58:46 |
| 3  | F-0003-2023   | Sunrise Valley Tahini, 6-pack     | 4057           | 173     | 2026-10-02 16:58:46 |
| 4  | F-0004-2026   | True North Enoki Mushrooms, 12 oz | 2076           | 12      | 2026-10-02 16:58:46 |
| 5  | F-0005-2025   | Mountain Fresh Whole Milk Yogurt, 1 | 9995         | 120     | 2026-10-02 16:58:46 |
```

Figure 1.1. MySQL tables firms and recall_notices (make db-show)

1.2 API

All endpoints are in code/auth_app/routers/firms.py and notices.py, need the HW4 session cookie (401 without it), and validate requests and responses with the Pydantic schemas in code/auth_app/schemas.py.

Method and path	Action	Success	Errors
POST /api/firms	create firm	201	409 contact_email in use, 422 invalid body
GET /api/firms?page=&page_size=	list firms, paged	200	
GET /api/firms/{id}	get one firm	200	404
PUT /api/firms/{id}	update firm	200	404, 409, 422
DELETE /api/firms/{id}	delete firm	204	404, 409 firm still has notices
POST /api/notices	create notice	201	409 recall_number in use, 422 invalid body or unknown firm_id
GET /api/notices?page=&page_size=	list notices, paged	200	
GET /api/notices/{id}	get one notice	200	404
PUT /api/notices/{id}	update notice	200	404, 409, 422
DELETE /api/notices/{id}	delete notice	204	404
GET /api/firms/{id}/notices	relationship query: all notices of one firm	200	404

code/auth_app/schemas.py — request schemas; the format of both unique fields is validated here

```
class FirmIn(BaseModel):

    name: str = Field(min_length=1, max_length=150)
    state: str = Field(pattern=r"^[A-Z]{2}$")          # two capital letters, e.g. CA
    contact_email: EmailStr                            # unique field, format checked here

class NoticeIn(BaseModel):

    product_name: str = Field(min_length=1, max_length=255)
    category: Category
    recall_number: str = Field(pattern=RECALL_NUMBER_PATTERN)     # unique field
    units_affected: int = Field(default=0, ge=0, le=INT_UNSIGNED_MAX)
    firm_id: int = Field(ge=1)                # required: every notice belongs to a firm
    recall_date: Optional[date] = None        # defaults to today on create
```

code/auth_app/routers/firms.py — the relationship query

```
@router.get("/{firm_id}/notices", response_model=FirmNoticesPage)
def list_firm_notices(firm_id: int, page: int = PageNo, page_size: int = PageSize,
                      db: Session = Depends(get_db)):
    firm = load_firm(db, firm_id)                                  # 404 if the firm is
unknown
    total = db.execute(select(func.count()).select_from(RecallNotice)
```

```
                        .where(RecallNotice.firm_id == firm_id)).scalar_one()
    stmt = (select(RecallNotice)
            .where(RecallNotice.firm_id == firm_id)
            .options(joinedload(RecallNotice.firm))            # needed: firm is
lazy="raise"
            .order_by(RecallNotice.recall_date.desc(), RecallNotice.id.desc())
            .limit(page_size).offset((page - 1) * page_size))
    items = db.execute(stmt).scalars().all()
    return FirmNoticesPage(firm=firm, page=page, page_size=page_size, total=total,
items=items)
```

code/auth_app/routers/firms.py — constraint violation becomes 409, not a crash

```
@router.delete("/{firm_id}", status_code=204)
def delete_firm(firm_id: int, db: Session = Depends(get_db)):
    firm = load_firm(db, firm_id)
    try:
        db.delete(firm)
        db.commit()
    except IntegrityError as exc:
        db.rollback()
        if mysql_code(exc) != ROW_IS_REFERENCED:
            raise
        count = db.execute(select(func.count()).select_from(RecallNotice)
                           .where(RecallNotice.firm_id == firm_id)).scalar_one()
        raise HTTPException(
            status_code=409,
            detail=f"Firm {firm_id} still has {count} recall notice(s); "
                   f"delete or reassign them first")
    return Response(status_code=204)
```

Postman collection: reports/hw05/postman_collection.json (19 requests with status checks). Figures 1.2–1.15 show each action and one example of each error class.

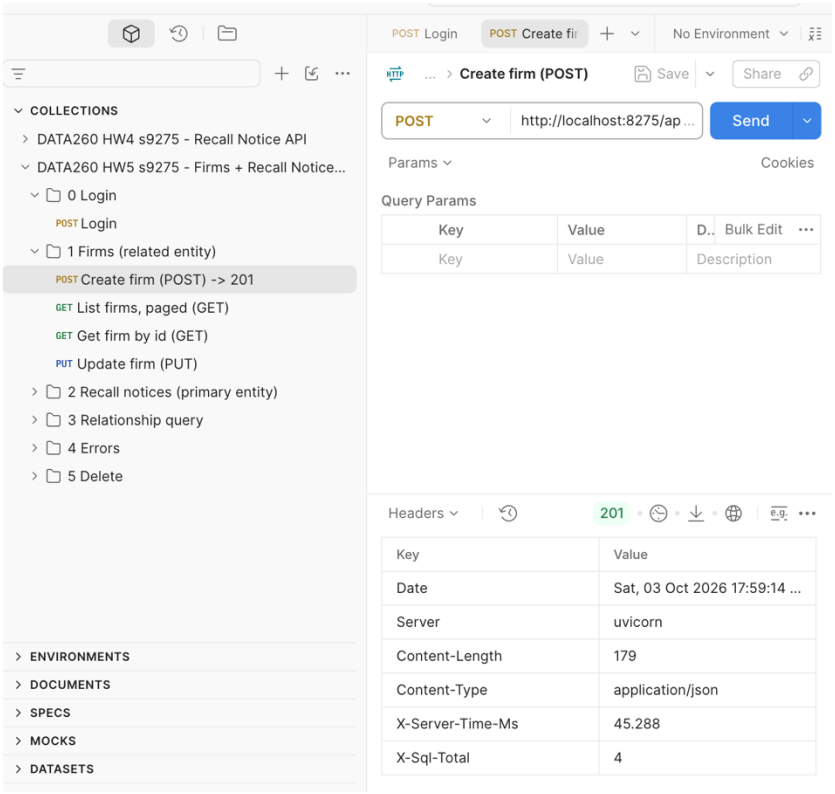


Figure 1.2. Postman: POST /api/firms → 201 Created

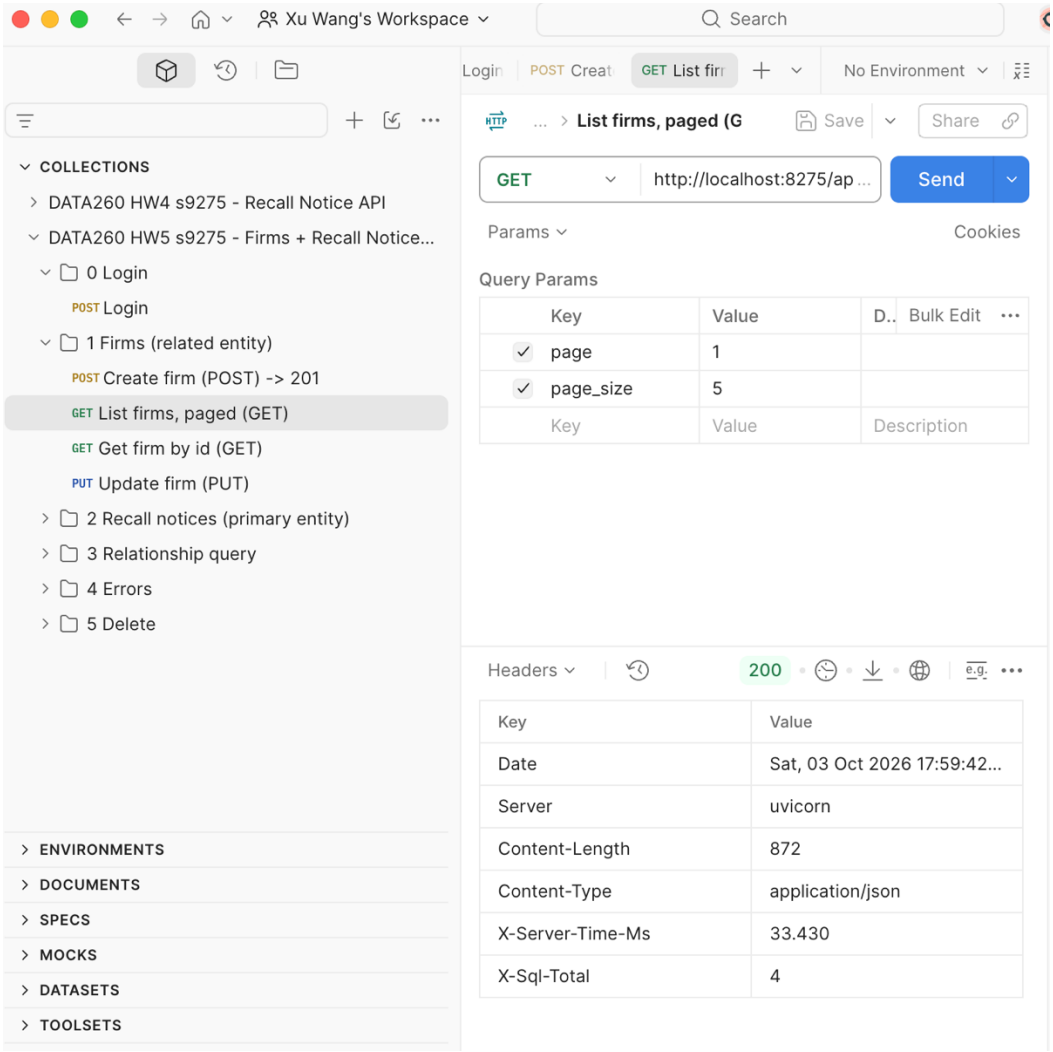


Figure 1.3. Postman: GET /api/firms?page=1&page_size=5 → 200

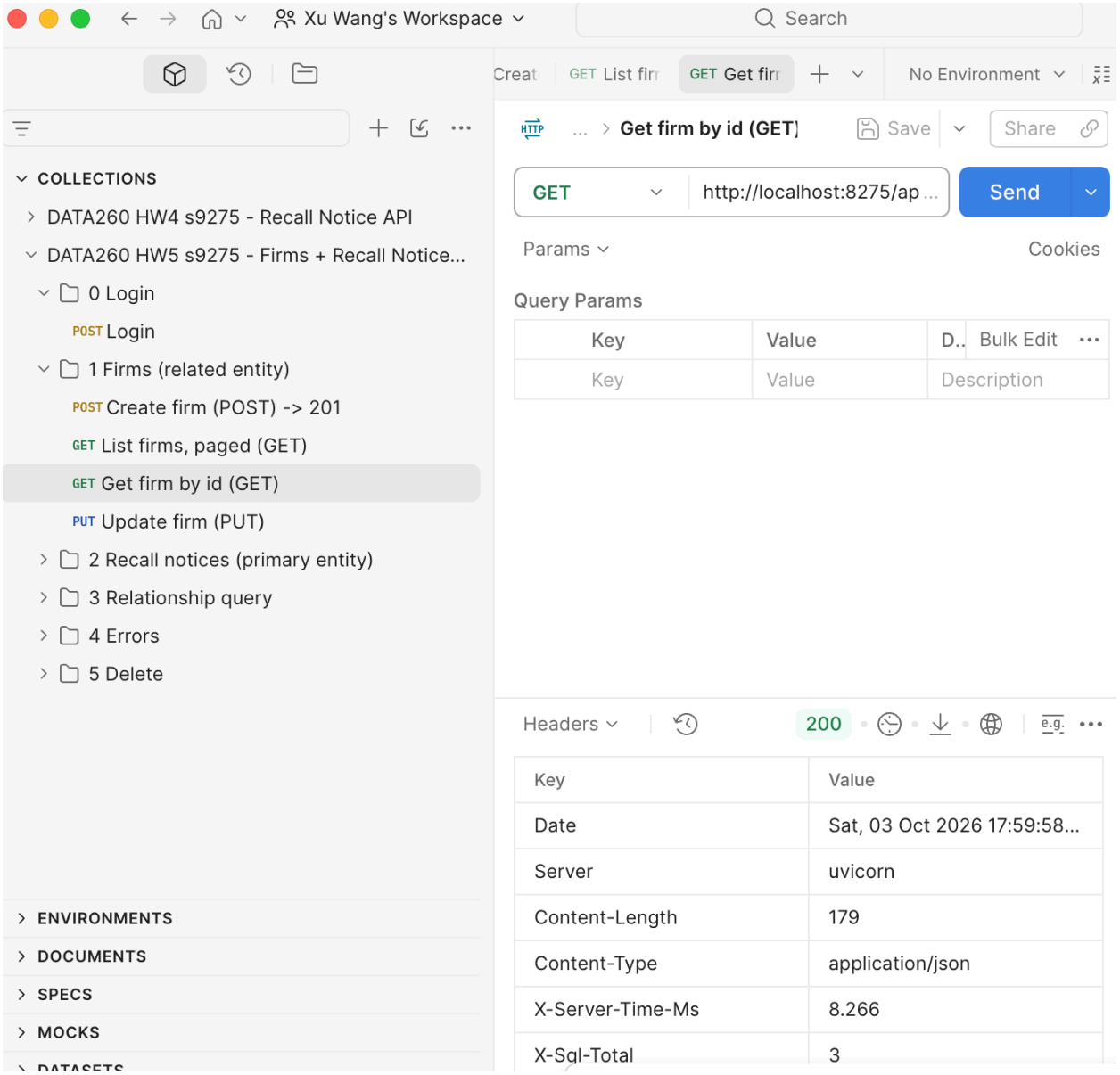


Figure 1.4. Postman: GET /api/firms/{id} → 200

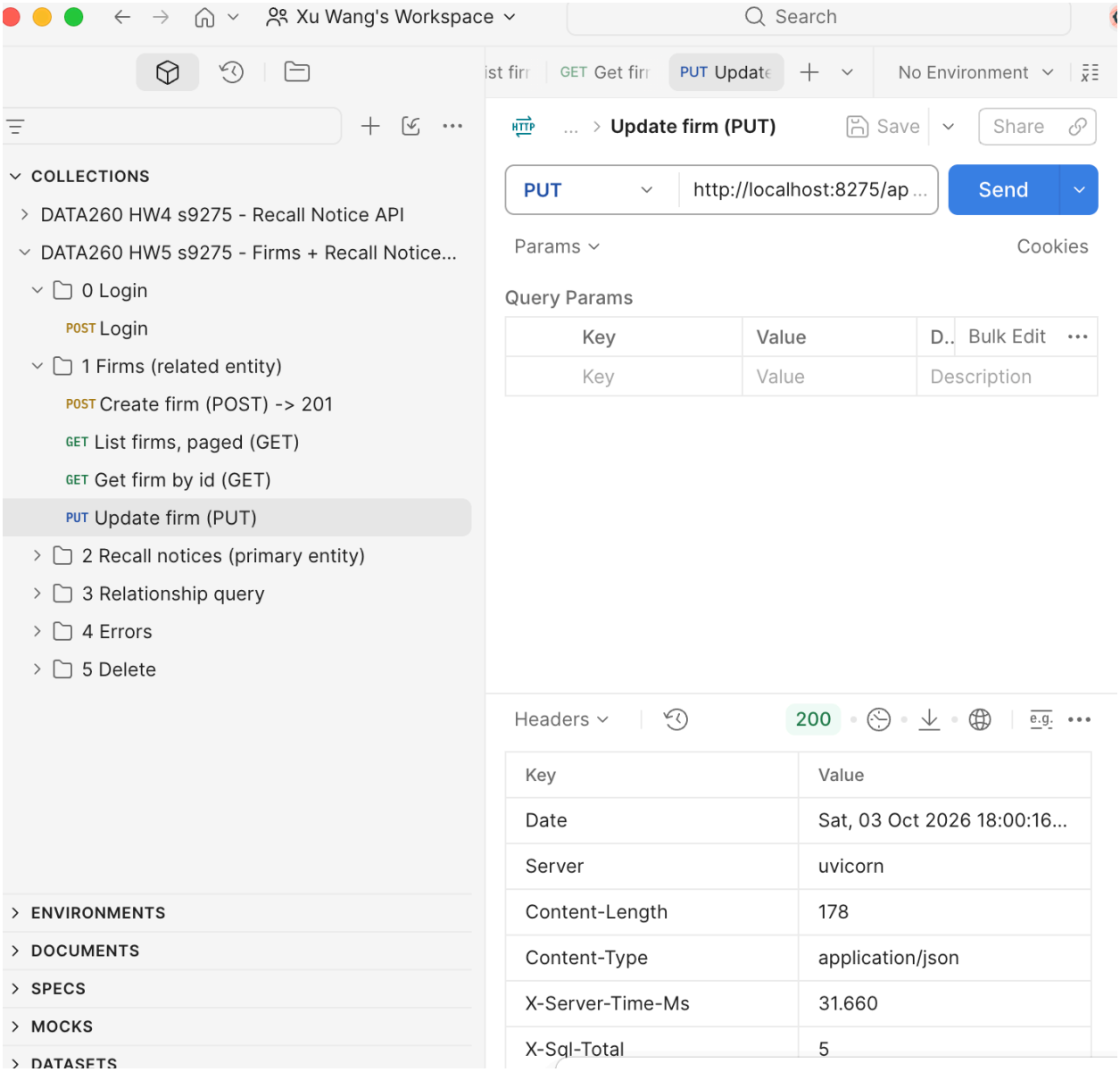


Figure 1.5. Postman: PUT /api/firms/{id} → 200

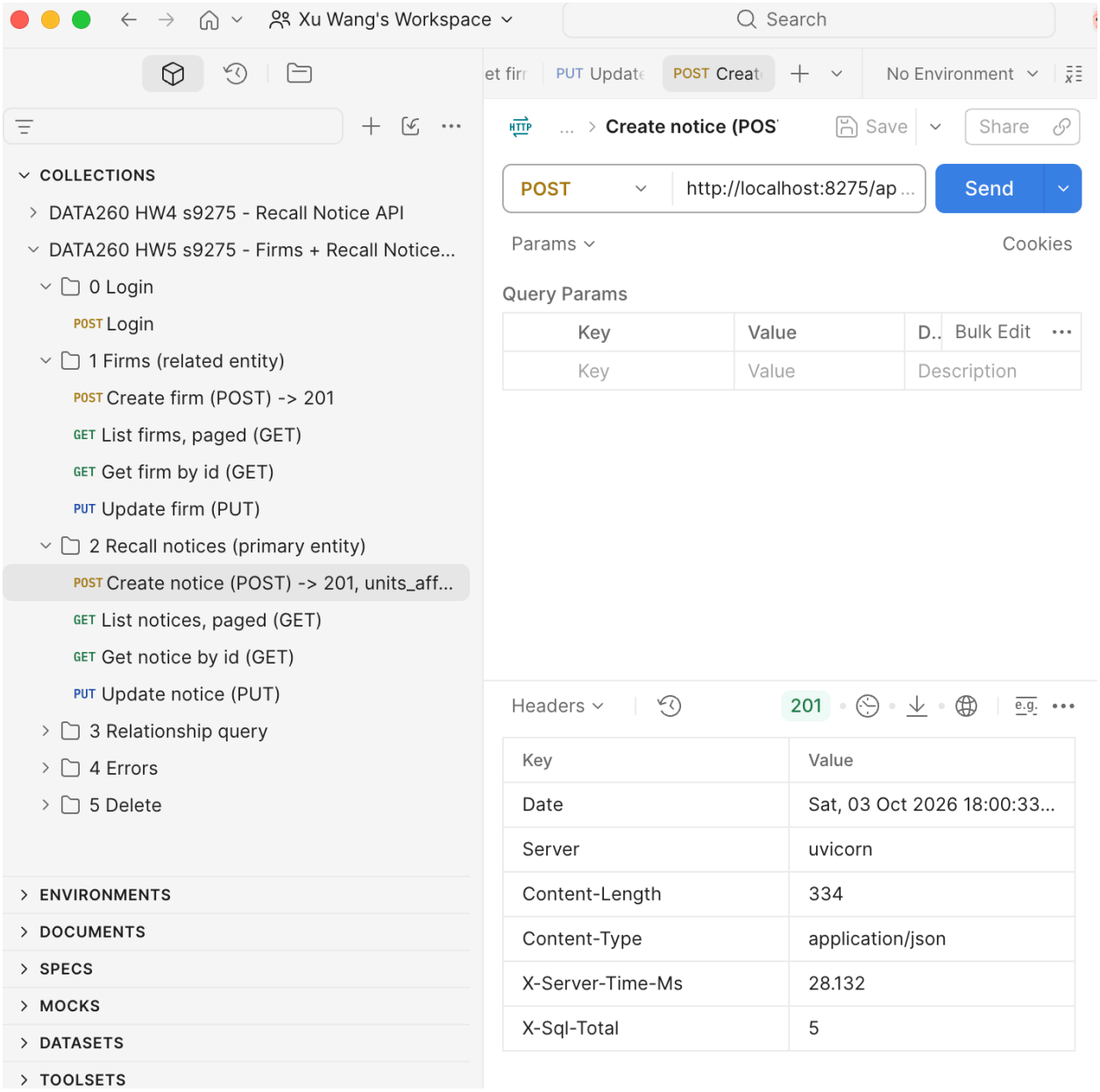


Figure 1.6. Postman: POST /api/notices → 201 Created

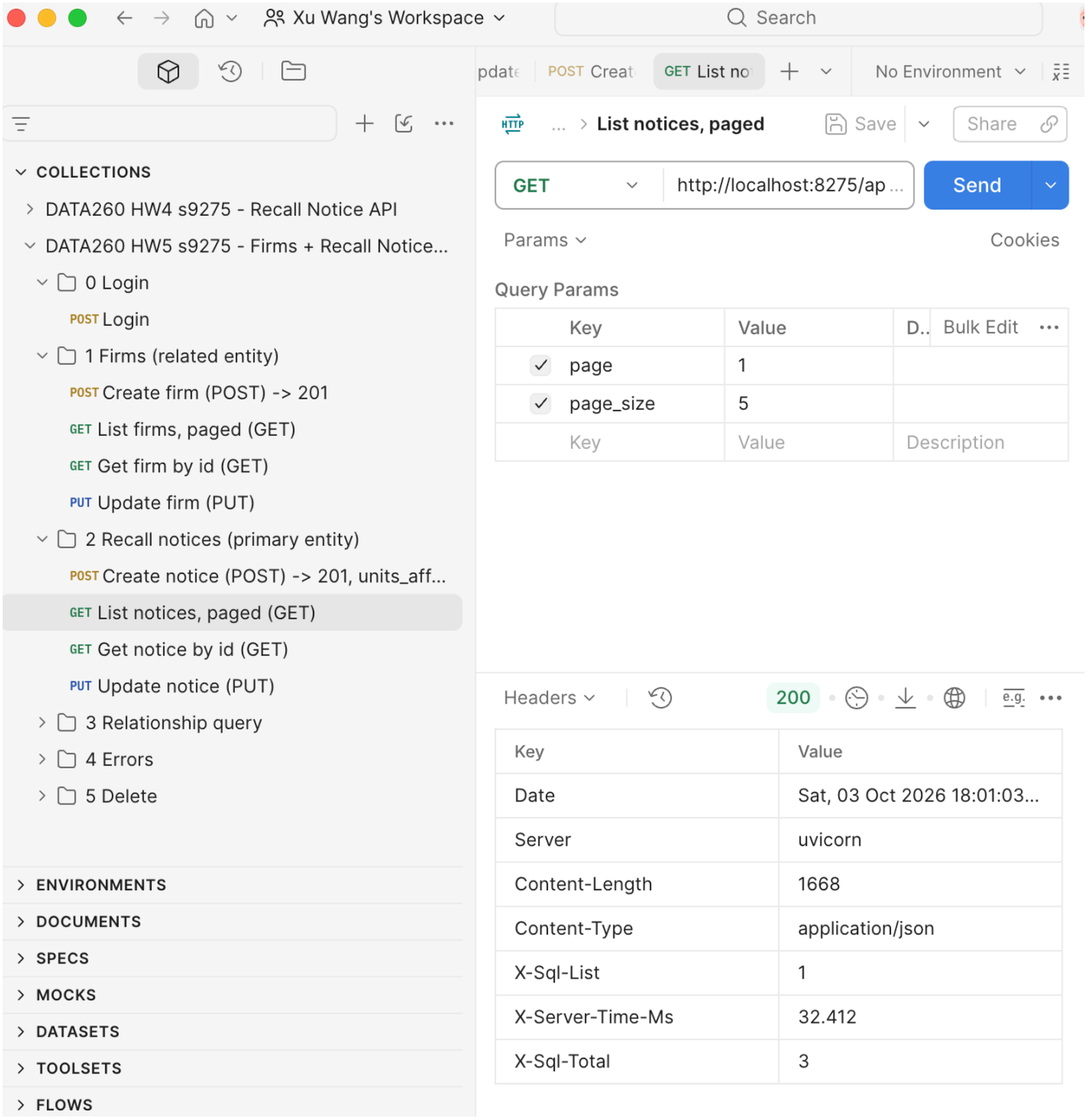


Figure 1.7. Postman: GET /api/notices?page=1&page_size=5 → 200

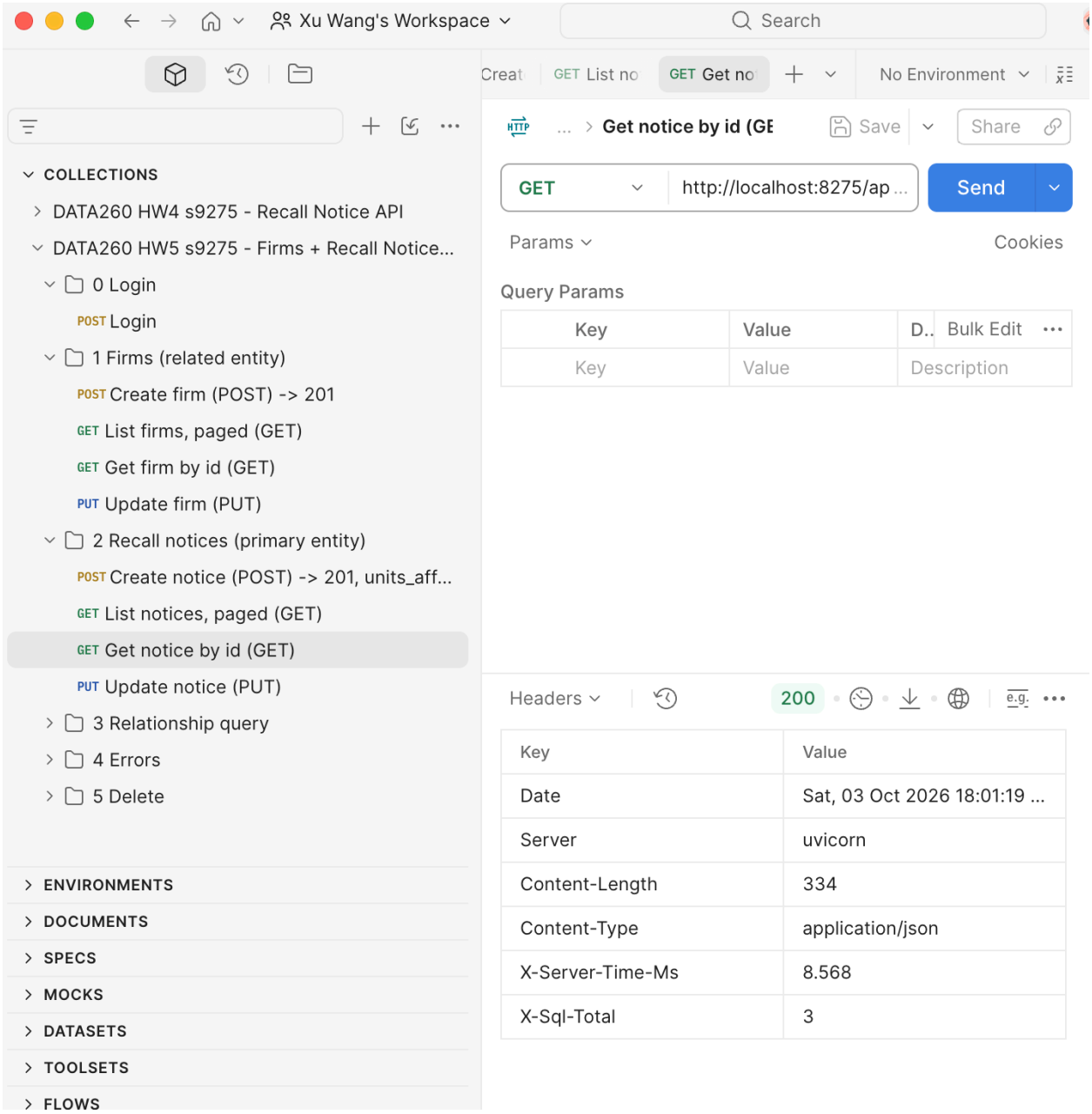


Figure 1.8. Postman: GET /api/notices/{id} → 200

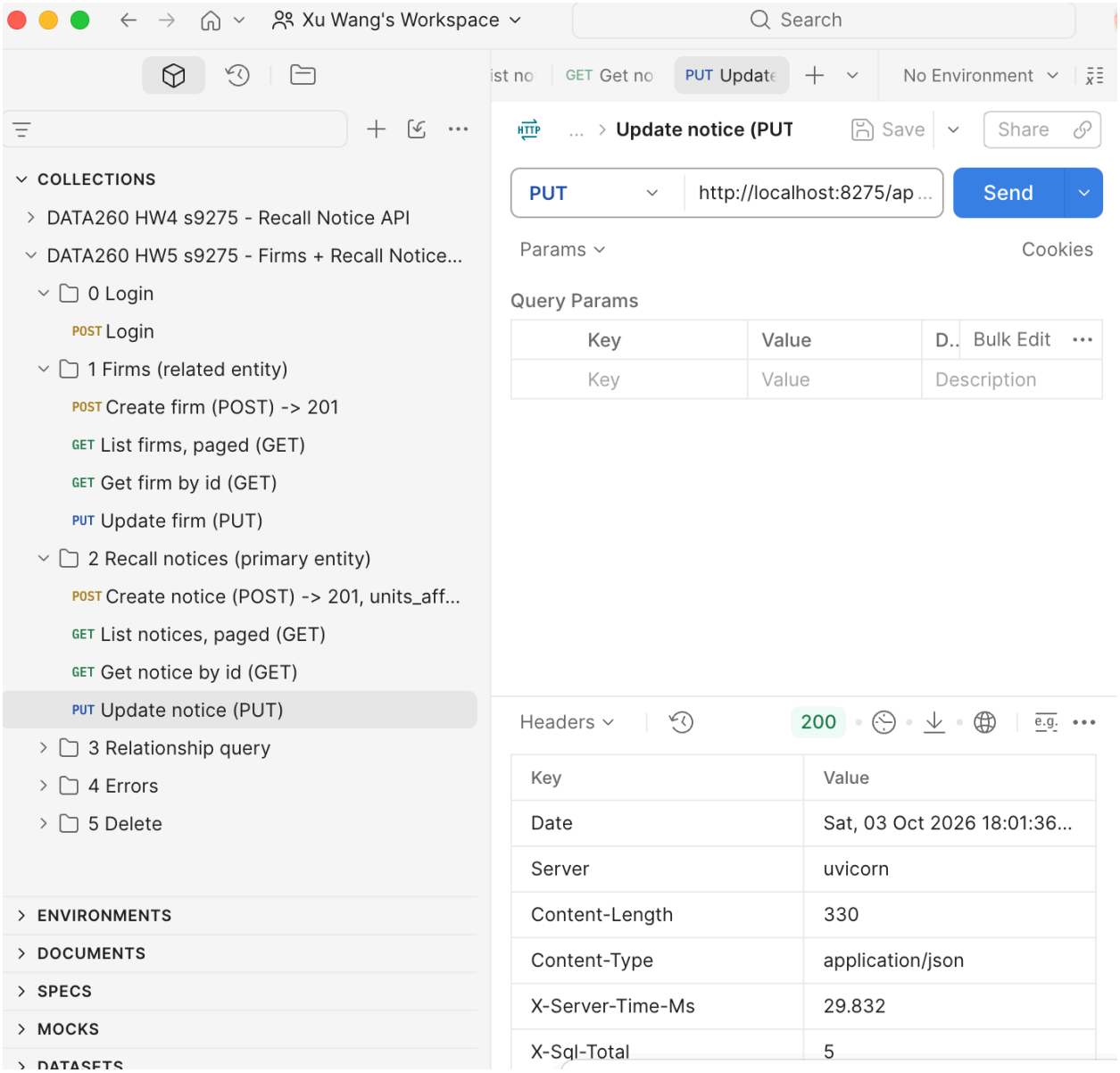


Figure 1.9. Postman: PUT /api/notices/{id} → 200

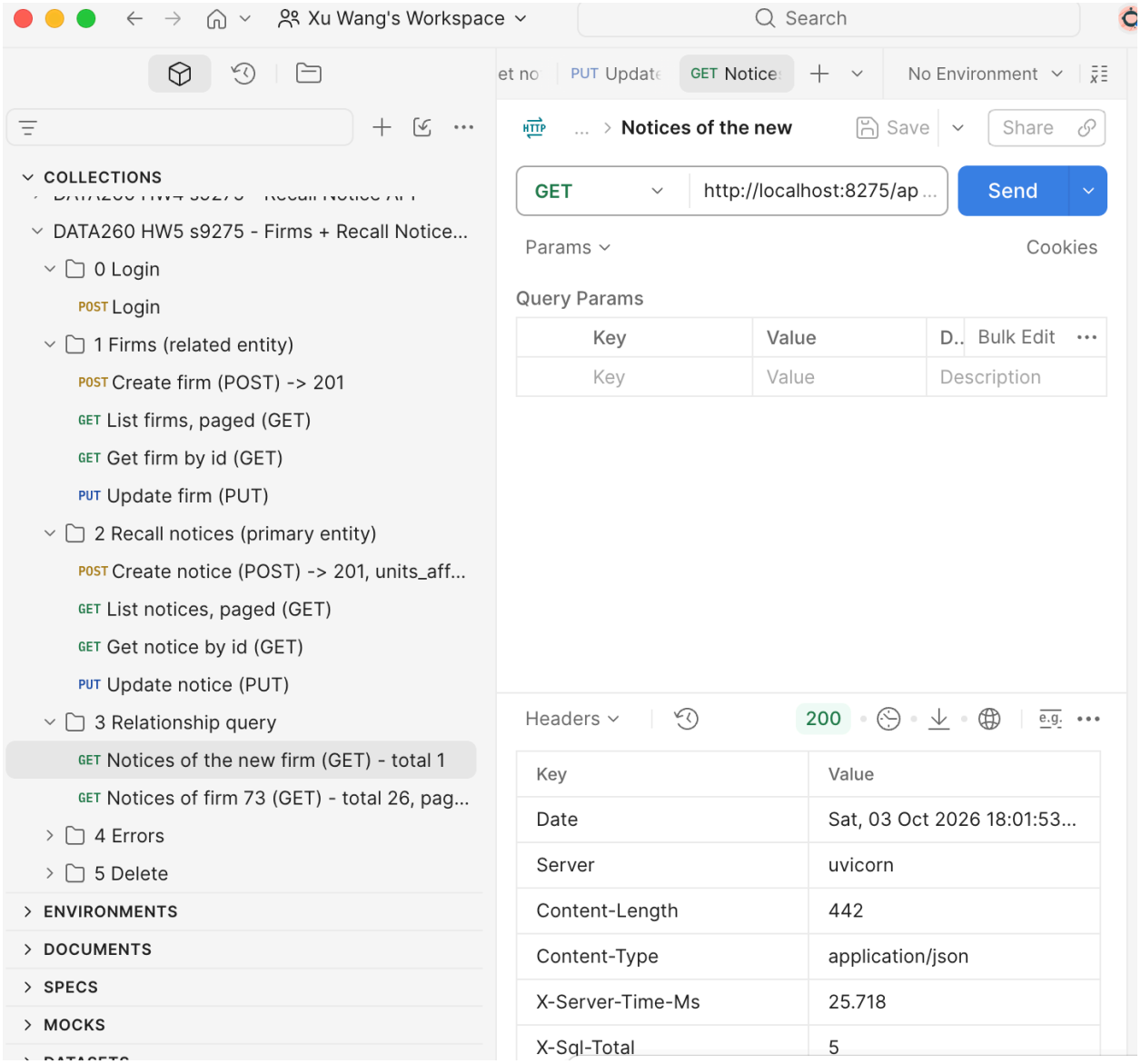


Figure 1.10. Postman: GET /api/firms/{id}/notices → 200 (relationship query)

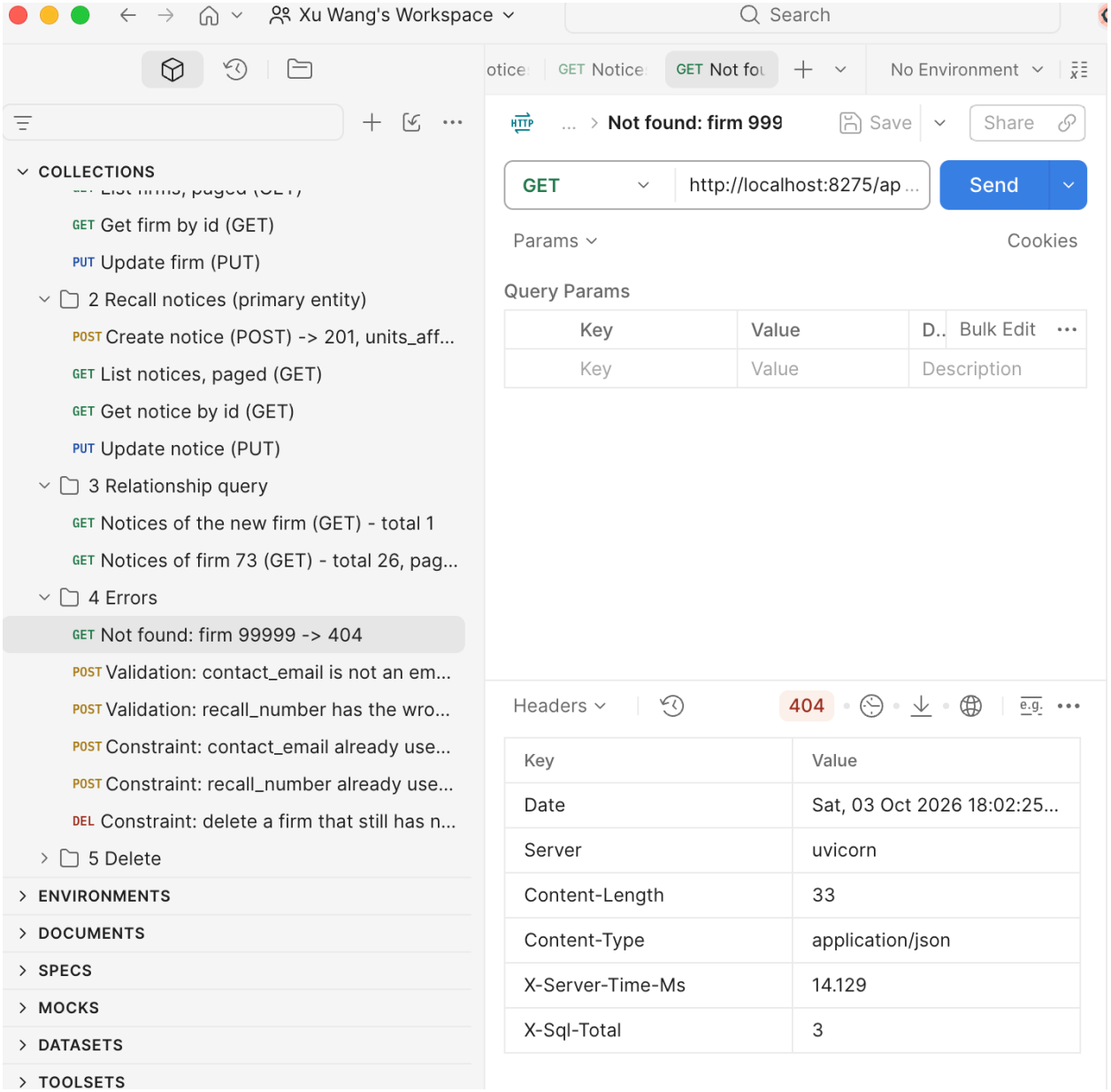


Figure 1.11. Postman: GET /api/firms/99999 → 404 Not Found

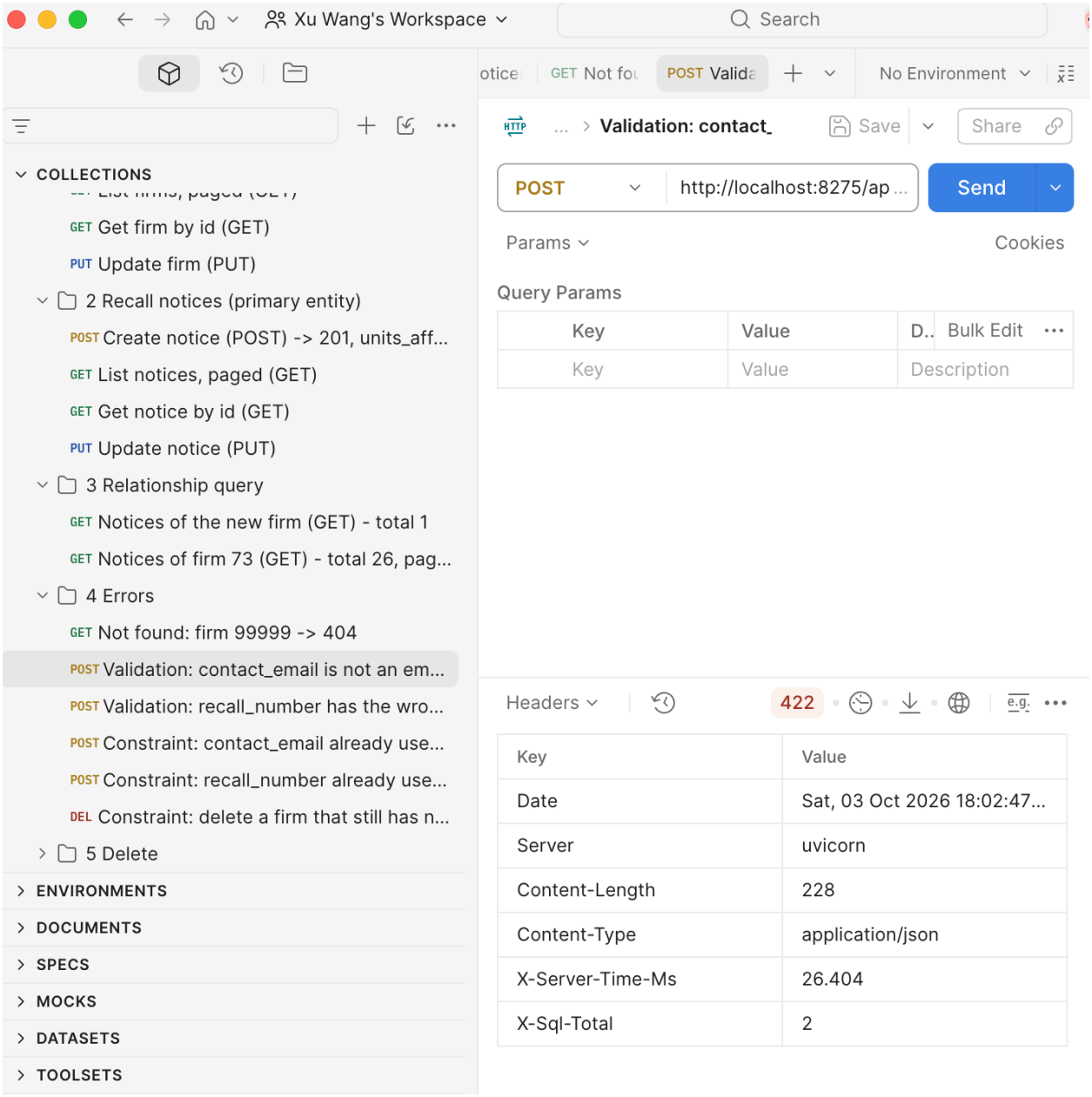


Figure 1.12. Postman: POST /api/firms with an invalid contact_email → 422

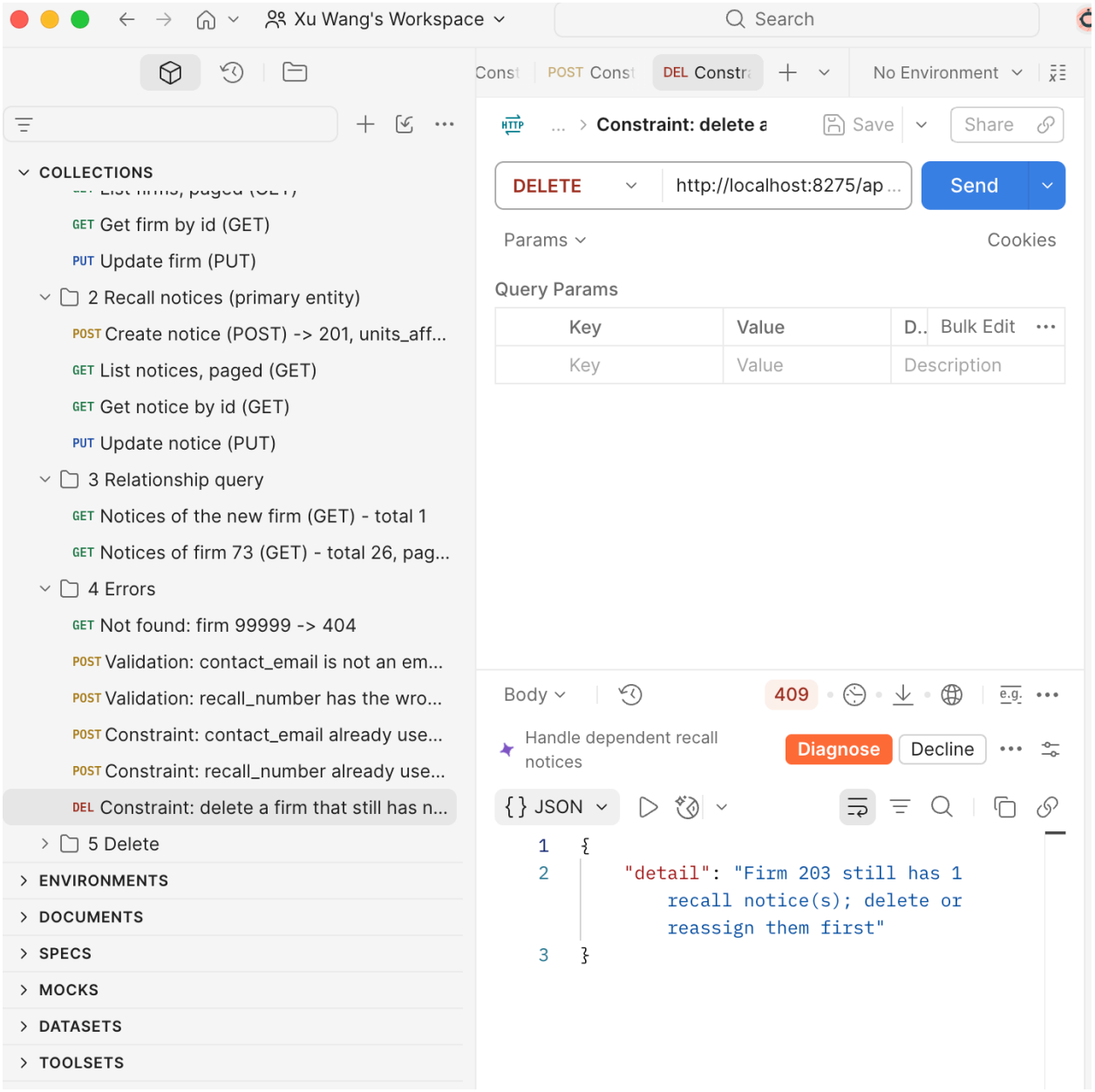


Figure 1.13. Postman: DELETE /api/firms/{id} while it still has notices → 409 Conflict

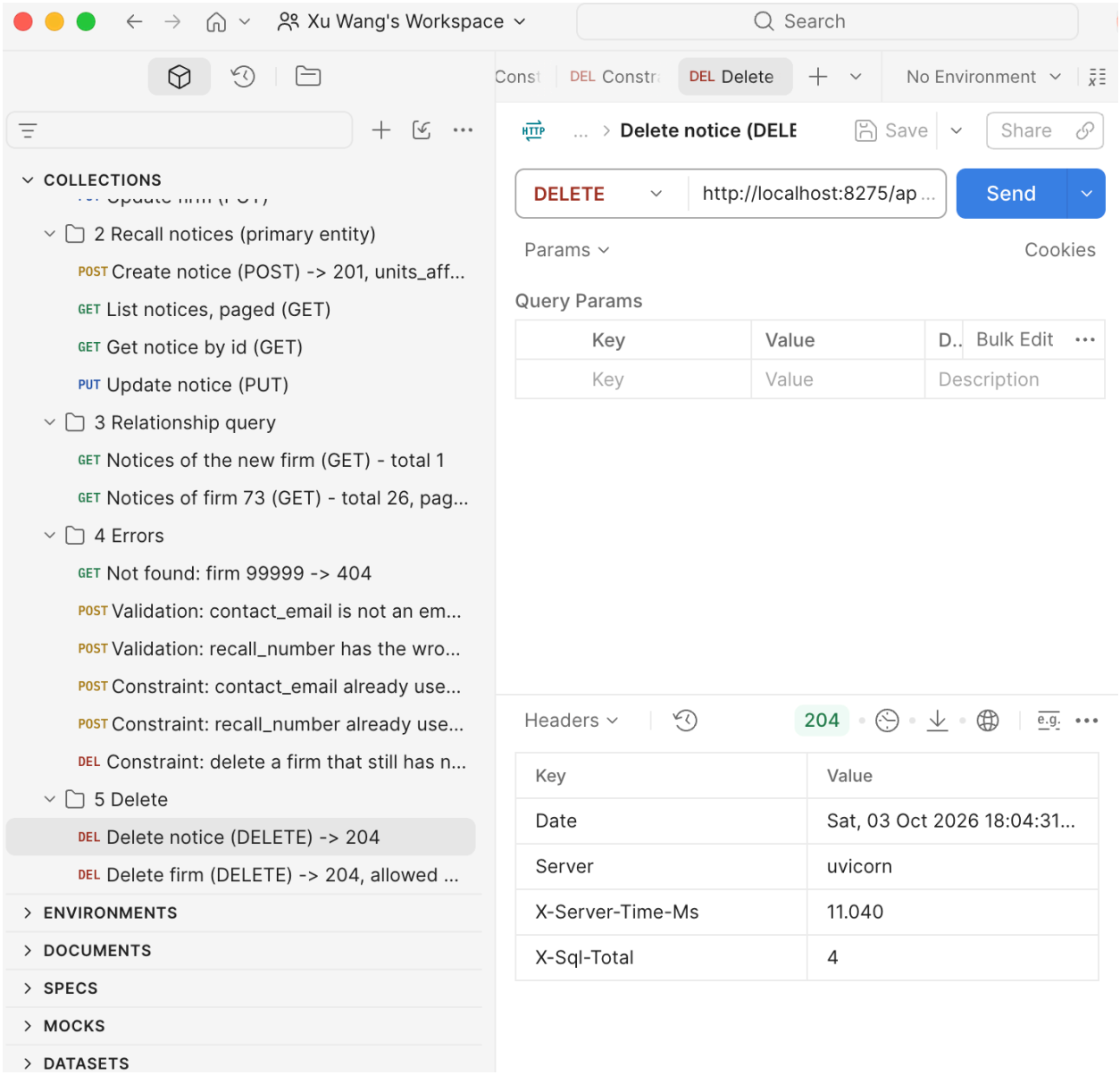


Figure 1.14. Postman: DELETE /api/notices/{id} → 204 No Content

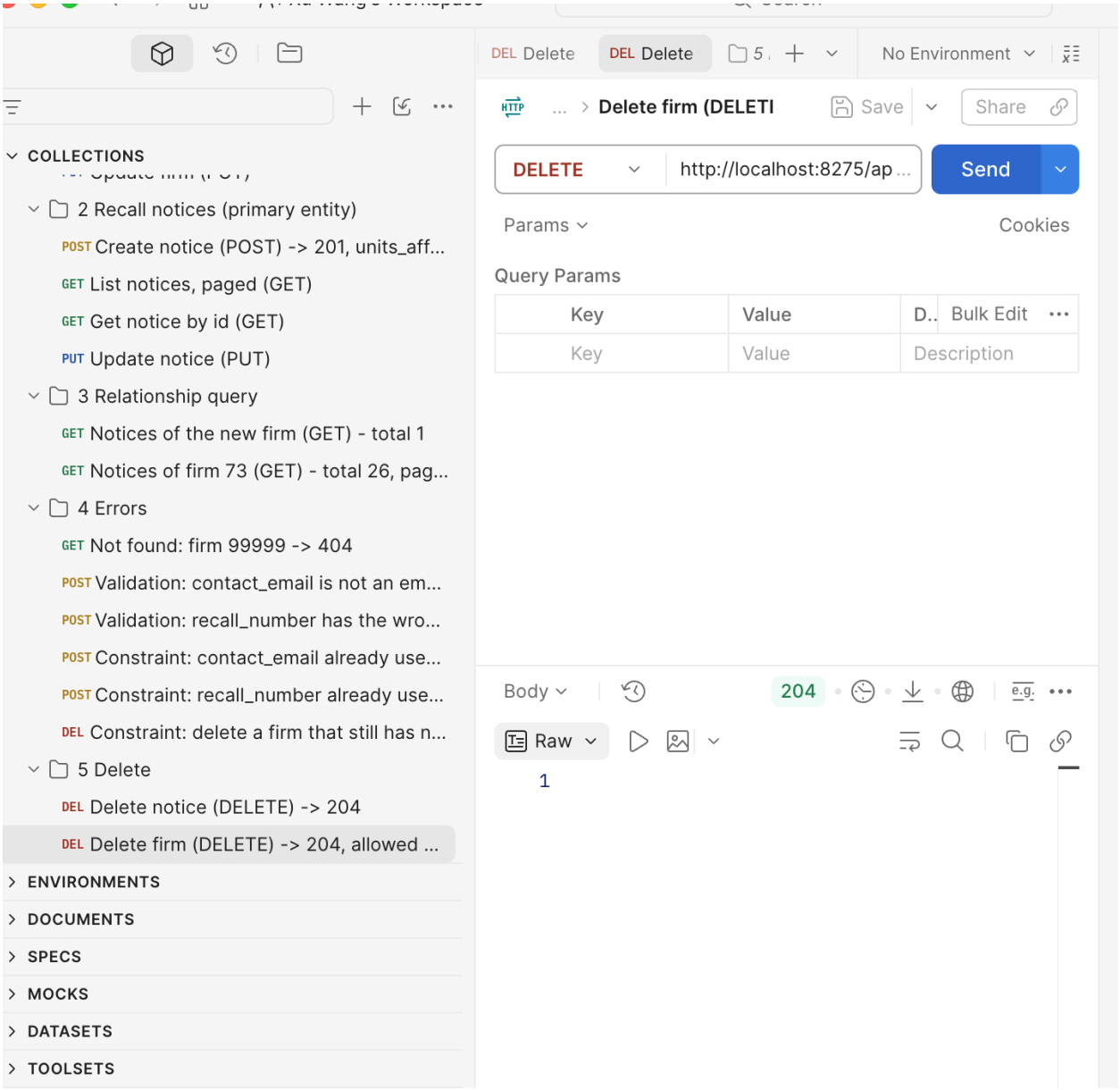


Figure 1.15. Postman: DELETE /api/firms/{id} → 204 No Content (allowed after its notice is gone)

1.3 Redux client

The React data layer for notices is now Redux Toolkit. Pages do not call the backend: they dispatch a thunk and read the list from the store with useSelector. The four thunks in frontend/src/features/notices/noticesSlice.js call the API with Axios (frontend/src/api/axios.js, withCredentials so the session cookie is sent) and return rejectWithValue({status, message}) on failure.

frontend/src/store/store.js — the store

```
export const store = configureStore({
  reducer: {
    notices: noticesReducer,
  },
});
```

frontend/src/features/notices/noticesSlice.js — reducers that keep the Home list in step with the backend

```
// adding: the new notice goes to the top of the list
.addCase(createNotice.fulfilled, (state, action) => {
  state.items.unshift(action.payload);
  if (state.items.length > PAGE_SIZE) state.items.pop();
})
// updating: swap the old copy for the one the backend returned
.addCase(updateNotice.fulfilled, (state, action) => {
  const index = state.items.findIndex((n) => n.id === action.payload.id);
  if (index !== -1) state.items[index] = action.payload;
})
// deleting: drop the record from the list
.addCase(deleteNotice.fulfilled, (state, action) => {
  state.items = state.items.filter((n) => n.id !== action.payload);
  state.error = null;
})
```

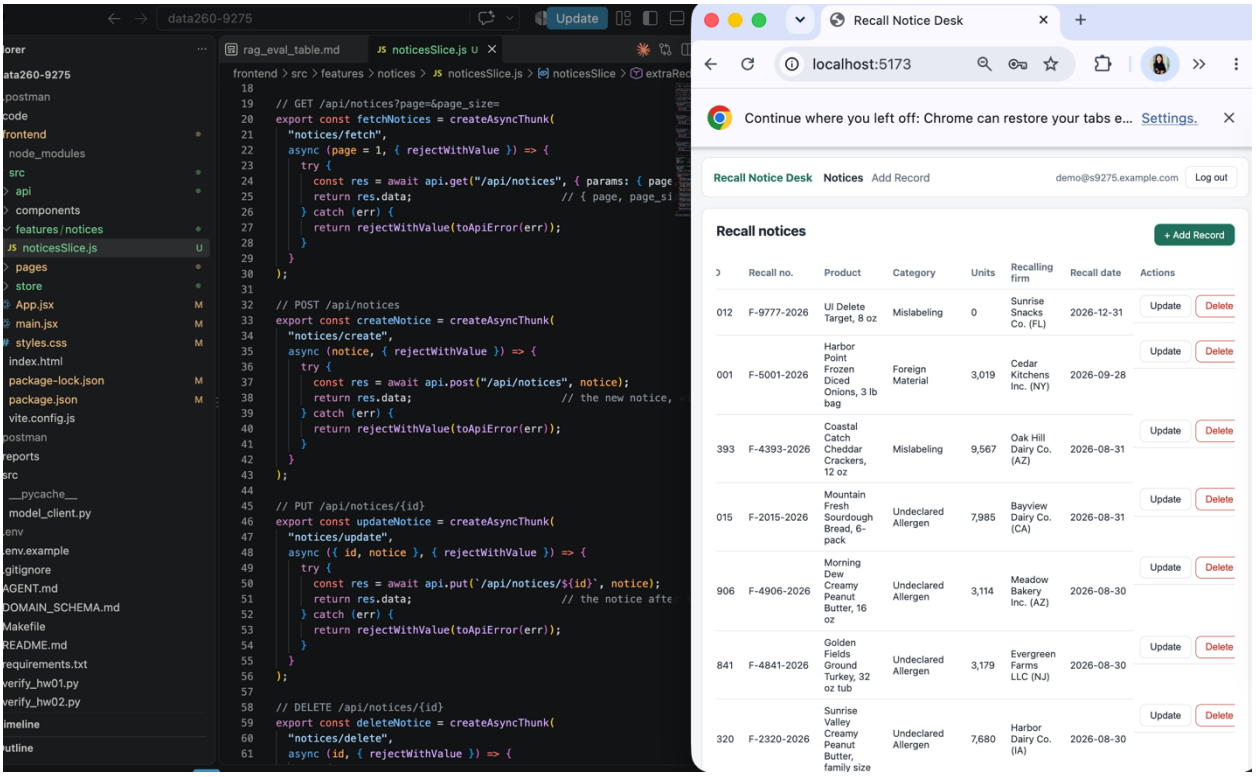


Figure 1.16. Redux: Home — list read from Redux state

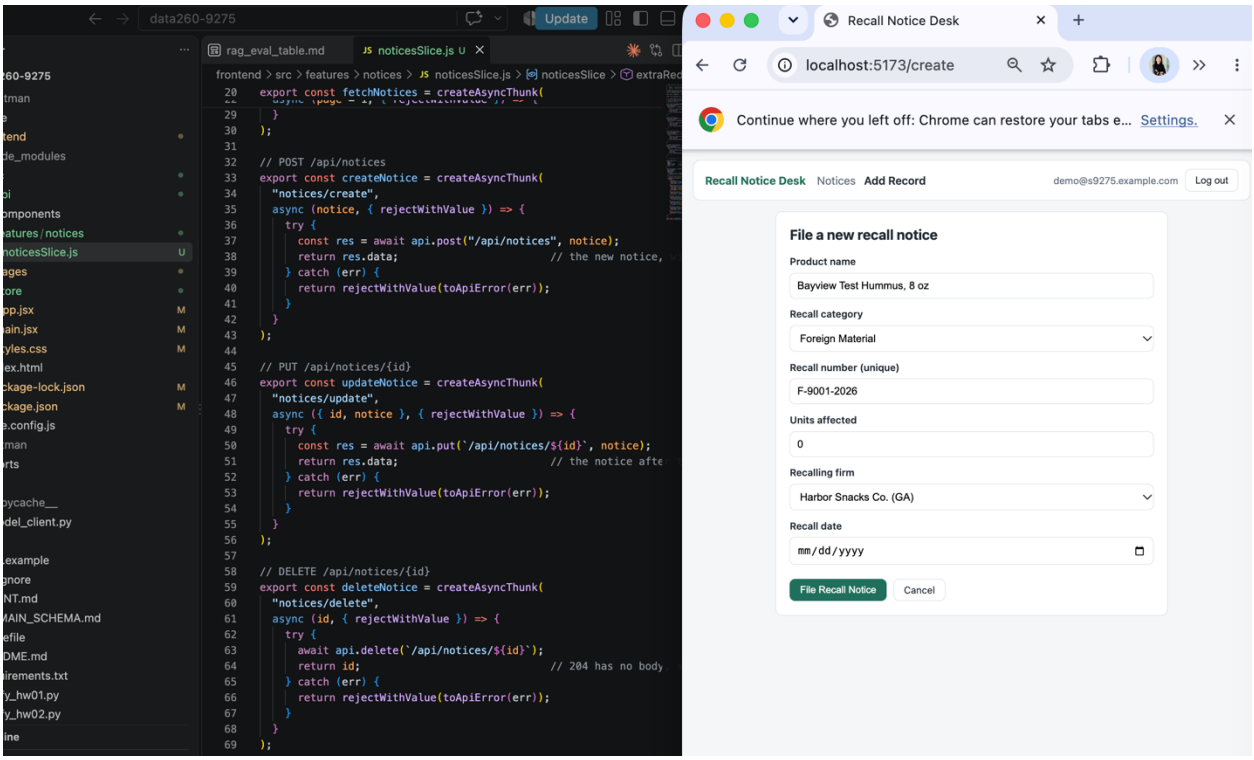


Figure 1.17. Redux: Create — createNotice thunk

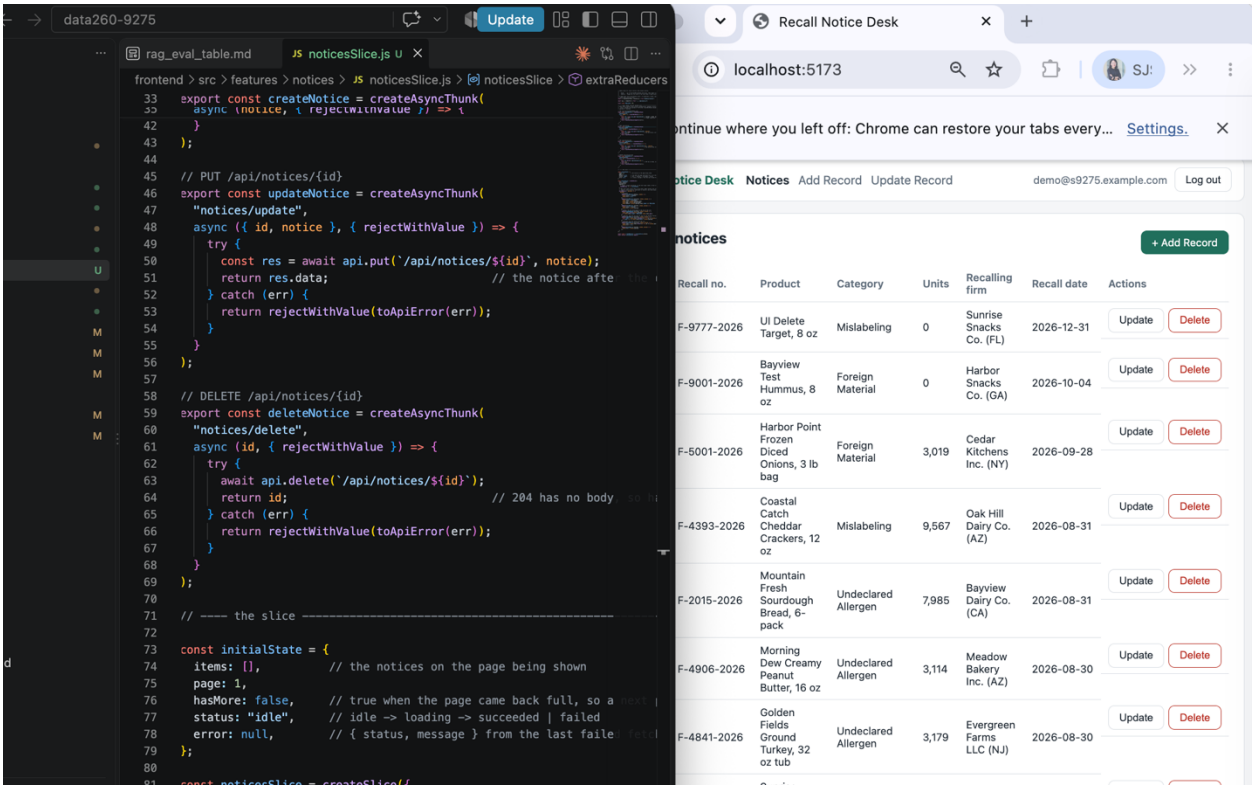


Figure 1.18. Redux: Update — updateNotice thunk

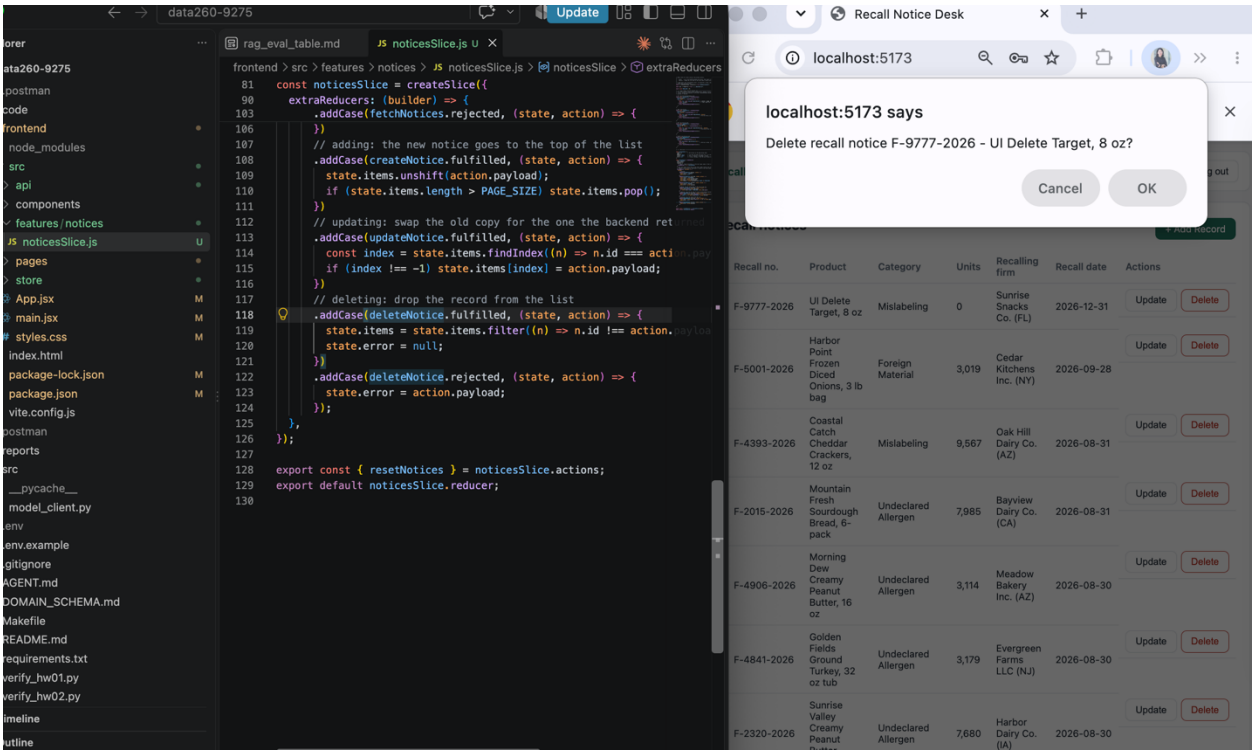


Figure 1.19. Redux: Delete — deleteNotice thunk

Part 2 — MCP tool servers

2A. TheMealDB server (meals)

code/meals_server.py is one file: FastMCP("meals") over STDIO with four tools. Logging goes to stderr only, because stdout carries the JSON-RPC messages. A {"meals": null} answer becomes an empty result with the message "no matches"; a network or JSON problem raises a short RuntimeError that the Inspector displays. limit is declared as 1–25, so an out-of-range value is rejected by the server.

Inspector start: make inspect-meals runs npx @modelcontextprotocol/inspector python code/meals_server.py. This is the same Inspector that mcp dev opens; mcp dev was not used because it runs the server through uv and installs the newest MCP SDK (2.x) instead of the pinned 1.x that this code imports.

code/meals_server.py — stderr logging, the shared fetch helper and one of the four tools

```
logging.basicConfig(stream=sys.stderr, level=logging.INFO,
                    format="%(asctime)s %(levelname)s meals: %(message)s")
log = logging.getLogger("meals")

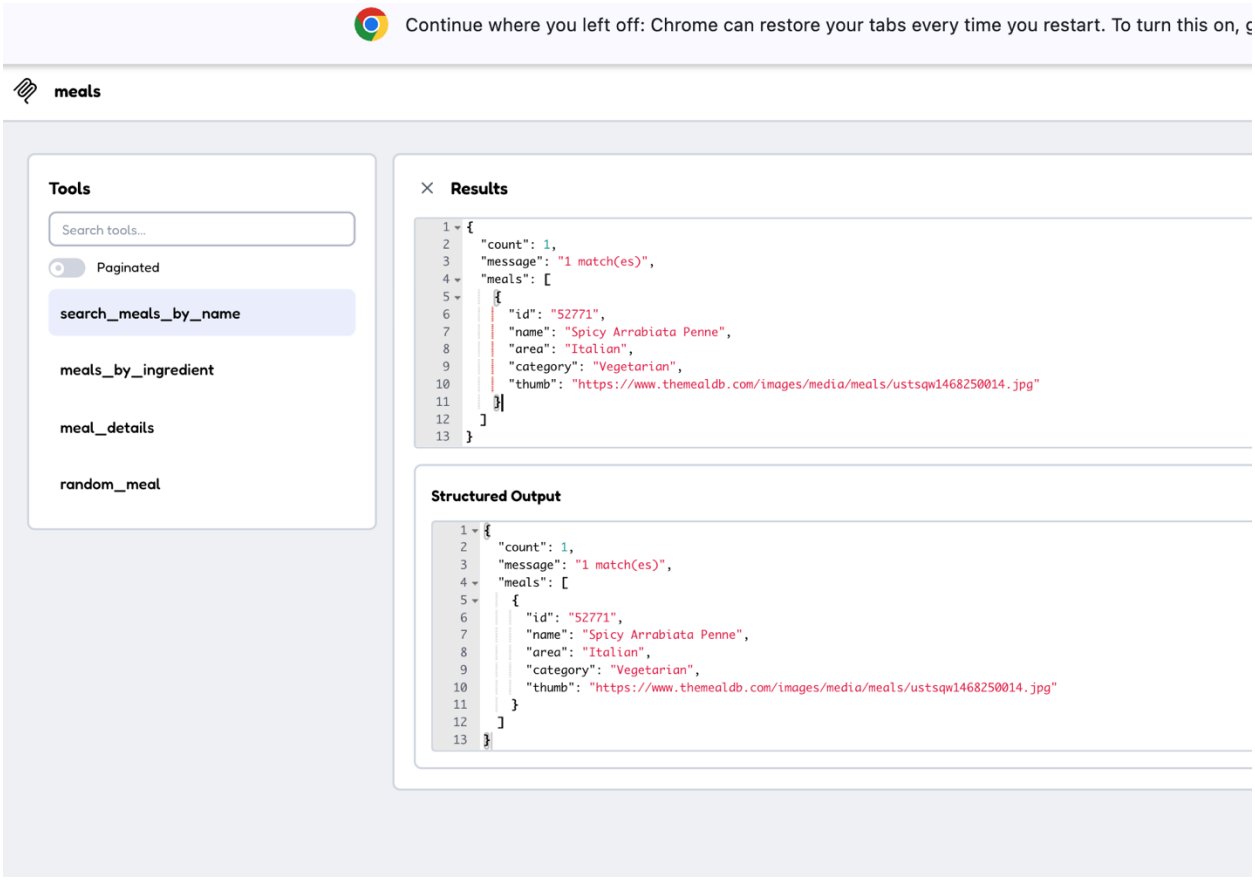
mcp = FastMCP("meals")


def fetch(endpoint: str, params: dict | None = None) -> list[dict]:
    url = f"{API_BASE}/{endpoint}"
    log.info("GET %s %s", url, params or "")
    try:
        response = httpx.get(url, params=params, timeout=TIMEOUT_SECONDS)
        response.raise_for_status()
```

```
        payload = response.json()
    except httpx.HTTPError as exc:
        log.error("request failed: %r", exc)
        raise RuntimeError(f"TheMealDB request failed: {exc}") from exc
    except ValueError as exc:
        log.error("response was not JSON")
        raise RuntimeError("TheMealDB returned a response that is not JSON") from exc

    meals = payload.get("meals") if isinstance(payload, dict) else None
    return meals if isinstance(meals, list) else []

@mcp.tool()
def search_meals_by_name(
    query: Annotated[str, Field(min_length=1, description="Part of a meal name, e.g.
Arrabiata")],
    limit: Annotated[int, Field(ge=1, le=25, description="How many meals to return (1-
25)")] = 5,
) -> dict[str, Any]:
    meals = fetch("search.php", {"s": query})
    return result([
        {"id": m["idMeal"], "name": m["strMeal"], "area": m.get("strArea"),
         "category": m.get("strCategory"), "thumb": m.get("strMealThumb")}
        for m in meals[:limit]
    ])
```


Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on,
meals
Tools
Search tools...
Paginated
search_meals_by_name
meals_by_ingredient
meal_details
random_meal
Results
{
"count": 1,
"message": "1 match(es)",
"meals": [
{
"id": "52771",
"name": "Spicy Arrabiata Penne",
"area": "Italian",
"category": "Vegetarian",
"thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"
}
]
}
Structured Output
{
"count": 1,
"message": "1 match(es)",
"meals": [
{
"id": "52771",
"name": "Spicy Arrabiata Penne",
"area": "Italian",
"category": "Vegetarian",
"thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"
}
]
}

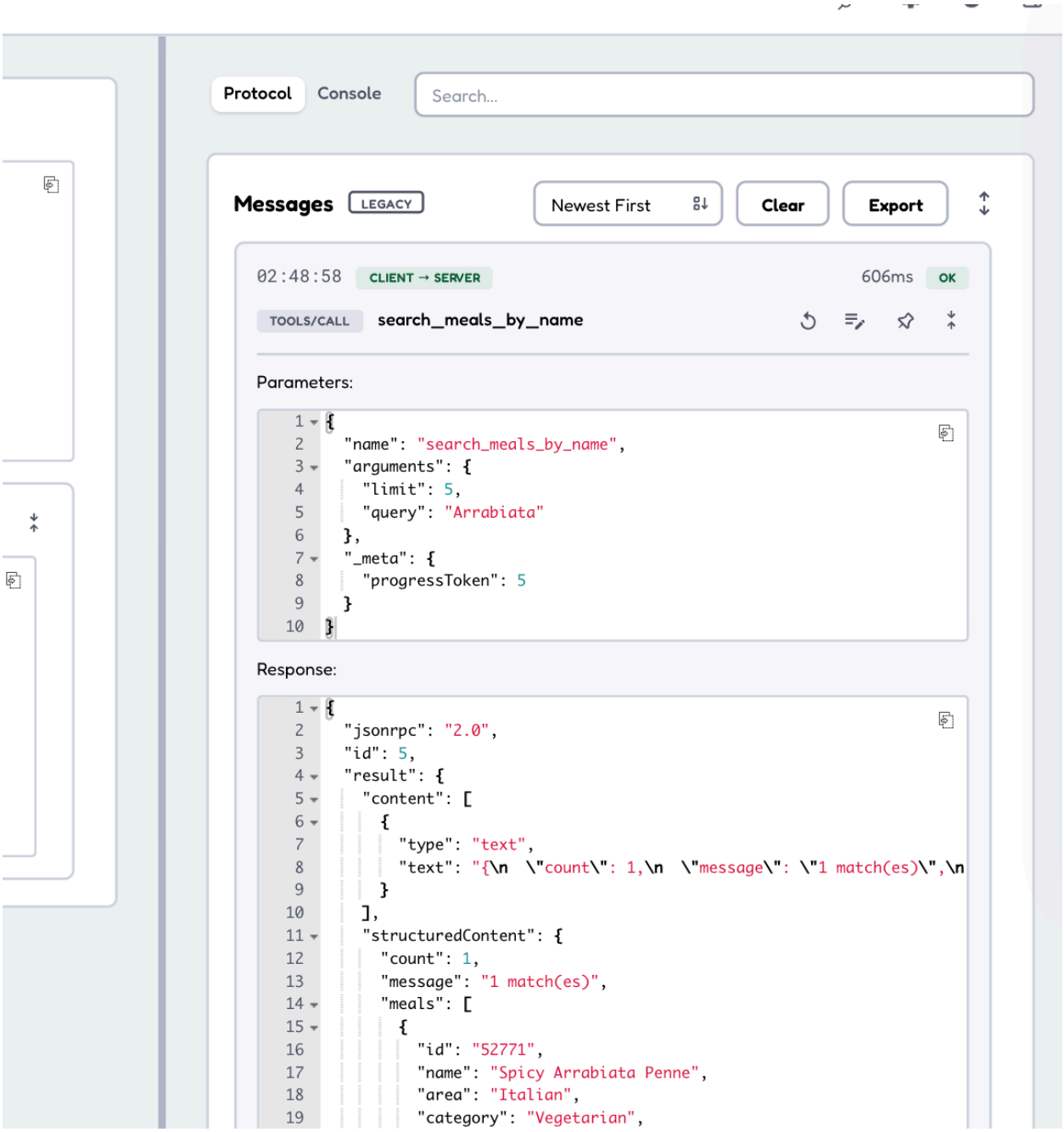
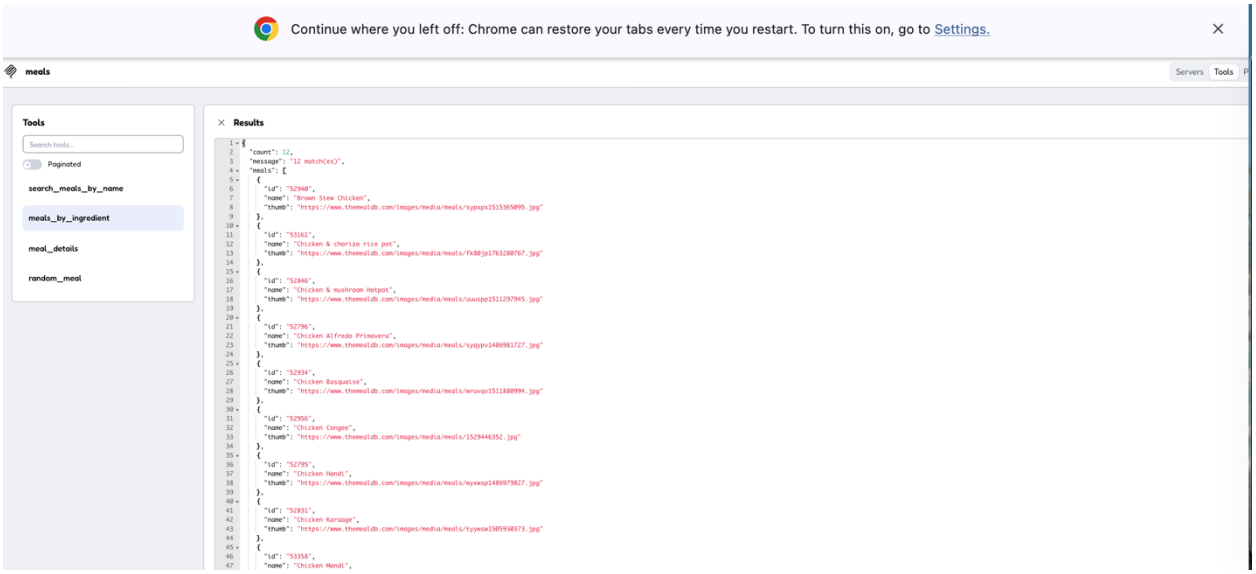


Figure 2.1. MCP Inspector: search_meals_by_name


Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Settings.
meals
Servers
Tools
Tools
Search tools...
Paginated
search_meals_by_name
meals_by_ingredient
meal_details
random_meal
Results
{
"count": 12,
"message": "12 match(es)",
"meals": [
{
"id": "52908",
"name": "Brown Stew Chicken",
"thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365095.jpg"
},
{
"id": "53161",
"name": "Chicken & chorizo rice pot",
"thumb": "https://www.themealdb.com/images/media/meals/fk80jp1763280767.jpg"
},
{
"id": "52846",
"name": "Chicken & mushroom Hotpot",
"thumb": "https://www.themealdb.com/images/media/meals/uuuspp1511297945.jpg"
},
{
"id": "52796",
"name": "Chicken Alfredo Primavera",
"thumb": "https://www.themealdb.com/images/media/meals/syqypv1486981727.jpg"
},
{
"id": "52934",
"name": "Chicken Basquaise",
"thumb": "https://www.themealdb.com/images/media/meals/wruvqv1511880994.jpg"
},
{
"id": "52956",
"name": "Chicken Congee",
"thumb": "https://www.themealdb.com/images/media/meals/1529446352.jpg"
},
{
"id": "52795",
"name": "Chicken Handi",
"thumb": "https://www.themealdb.com/images/media/meals/wyxwsp1486979827.jpg"
},
{
"id": "52831",
"name": "Chicken Karaage",
"thumb": "https://www.themealdb.com/images/media/meals/tyywsw1505930373.jpg"
},
{
"id": "53358",
"name": "Chicken Mandi",

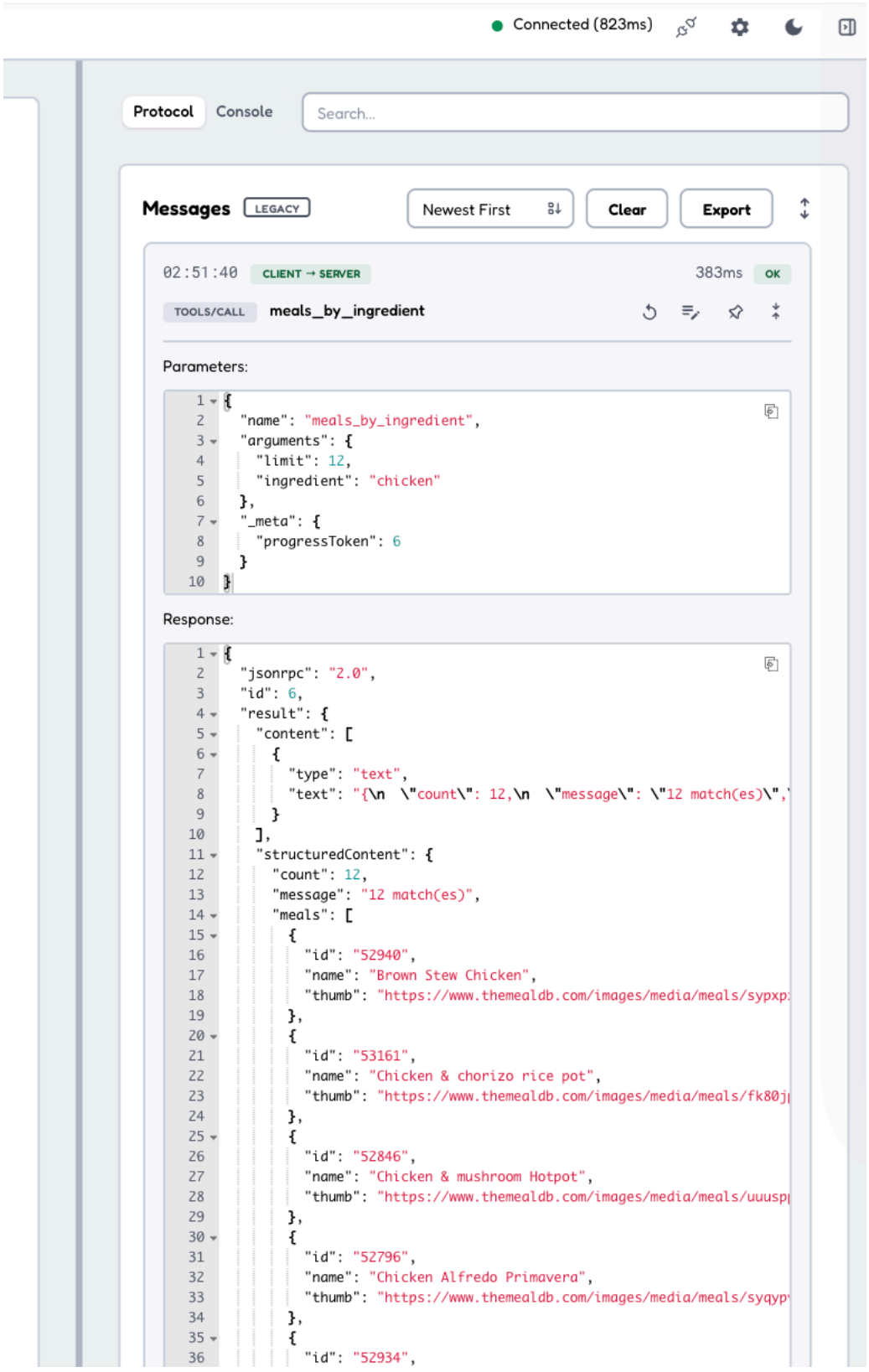
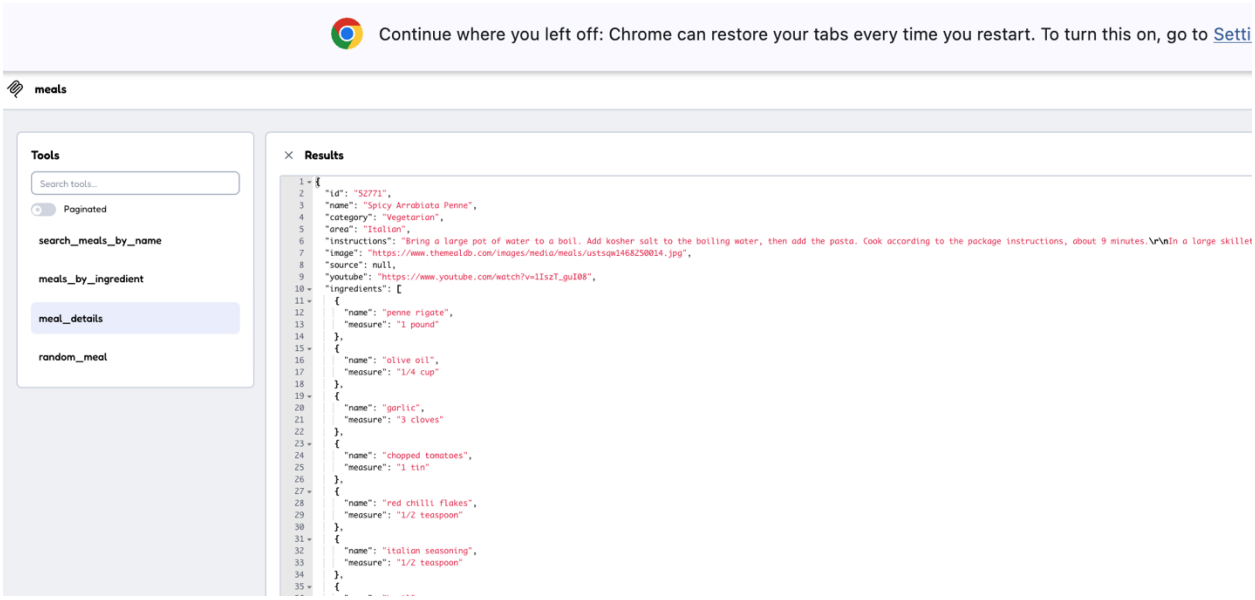


Figure 2.2. MCP Inspector: meals_by_ingredient


Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Setti
meals
Tools
Search tools...
Paginated
search_meals_by_name
meals_by_ingredient
meal_details
random_meal
Results
{
"id": "52771",
"name": "Spicy Arrabiata Penne",
"category": "Vegetarian",
"area": "Italian",
"instructions": "Bring a large pot of water to a boil. Add kosher salt to the boiling water, then add the pasta. Cook according to the package instructions, about 9 minutes.\r\nIn a large skillet
"image": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg",
"source": null,
"youtube": "https://www.youtube.com/watch?v=1IszT_guI08",
"ingredients": [
{
"name": "penne rigate",
"measure": "1 pound"
},
{
"name": "olive oil",
"measure": "1/4 cup"
},
{
"name": "garlic",
"measure": "3 cloves"
},
{
"name": "chopped tomatoes",
"measure": "1 tin"
},
{
"name": "red chilli flakes",
"measure": "1/2 teaspoon"
},
{
"name": "italian seasoning",
"measure": "1/2 teaspoon"
},
{

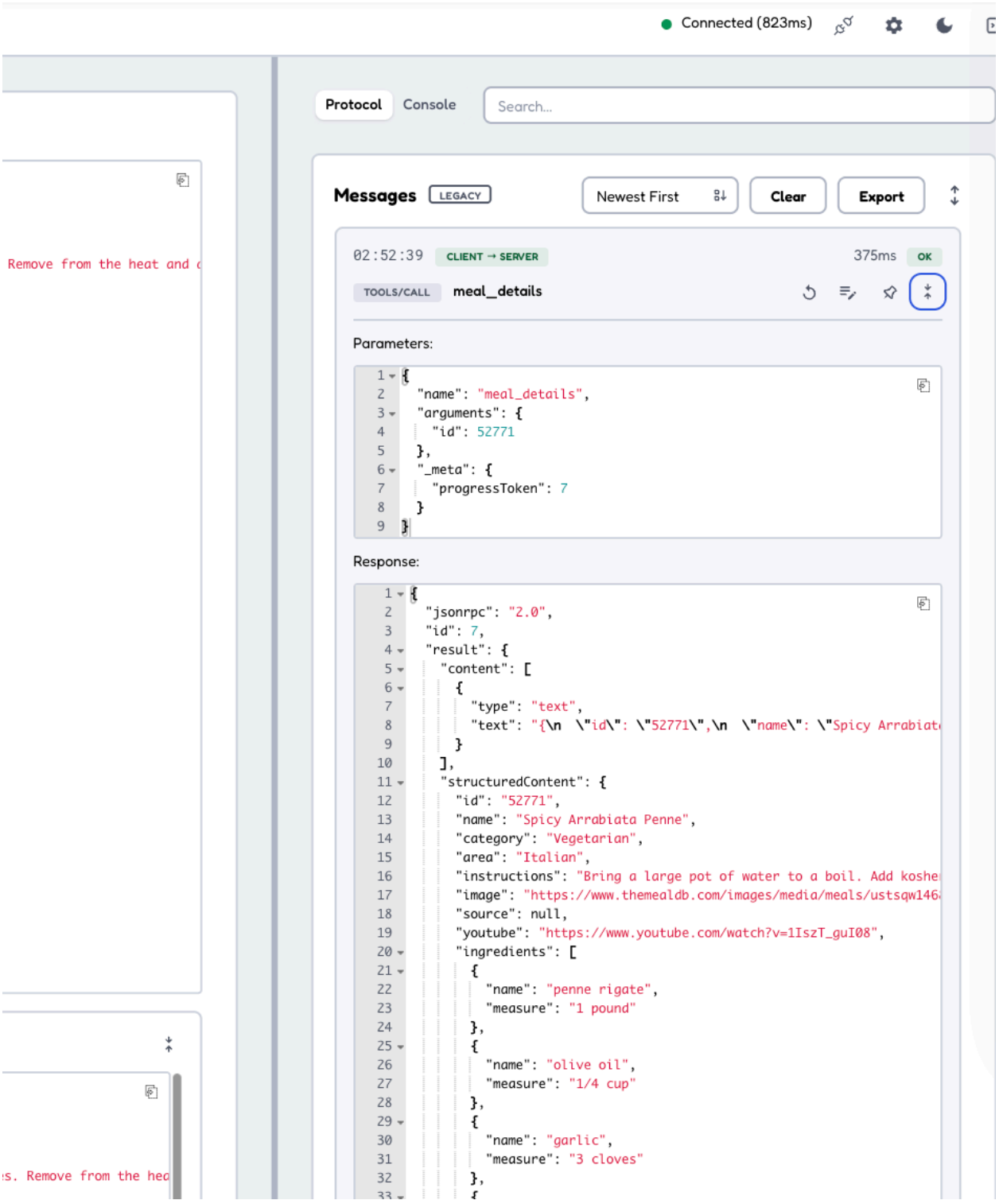
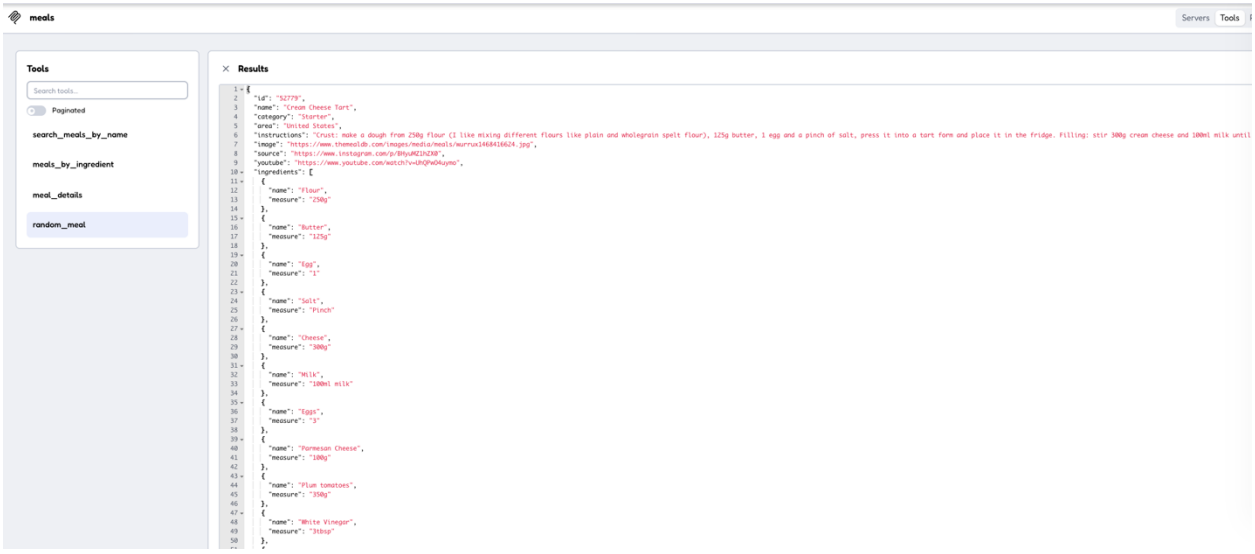


Figure 2.3. MCP Inspector: meal_details


meals
Servers
Tools
Tools
Search tools...
Paginated
search_meals_by_name
meals_by_ingredient
meal_details
random_meal
Results
{
"id": "52779",
"name": "Cream Cheese Tart",
"category": "Starter",
"area": "United States",
"instructions": "Crust: make a dough from 250g flour (I like mixing different flours like plain and wholegrain spelt flour), 125g butter, 1 egg and a pinch of salt, press it into a tart form and place it in the fridge. Filling: stir 300g cream cheese and 100ml milk until
"image": "https://www.themealdb.com/images/media/meals/wurrux1468416624.jpg",
"source": "https://www.instagram.com/p/BHyuMZ1hZXB/",
"youtube": "https://www.youtube.com/watch?v=UhQPwO4uymo",
"ingredients": [
{
"name": "Flour",
"measure": "250g"
},
{
"name": "Butter",
"measure": "125g"
},
{
"name": "Egg",
"measure": "1"
},
{
"name": "Salt",
"measure": "Pinch"
},
{
"name": "Cheese",
"measure": "300g"
},
{
"name": "Milk",
"measure": "100ml milk"
},
{
"name": "Eggs",
"measure": "3"
},
{
"name": "Parmesan Cheese",
"measure": "100g"
},
{
"name": "Plum tomatoes",
"measure": "350g"
},
{
"name": "White Vinegar",
"measure": "3tbsp"
},

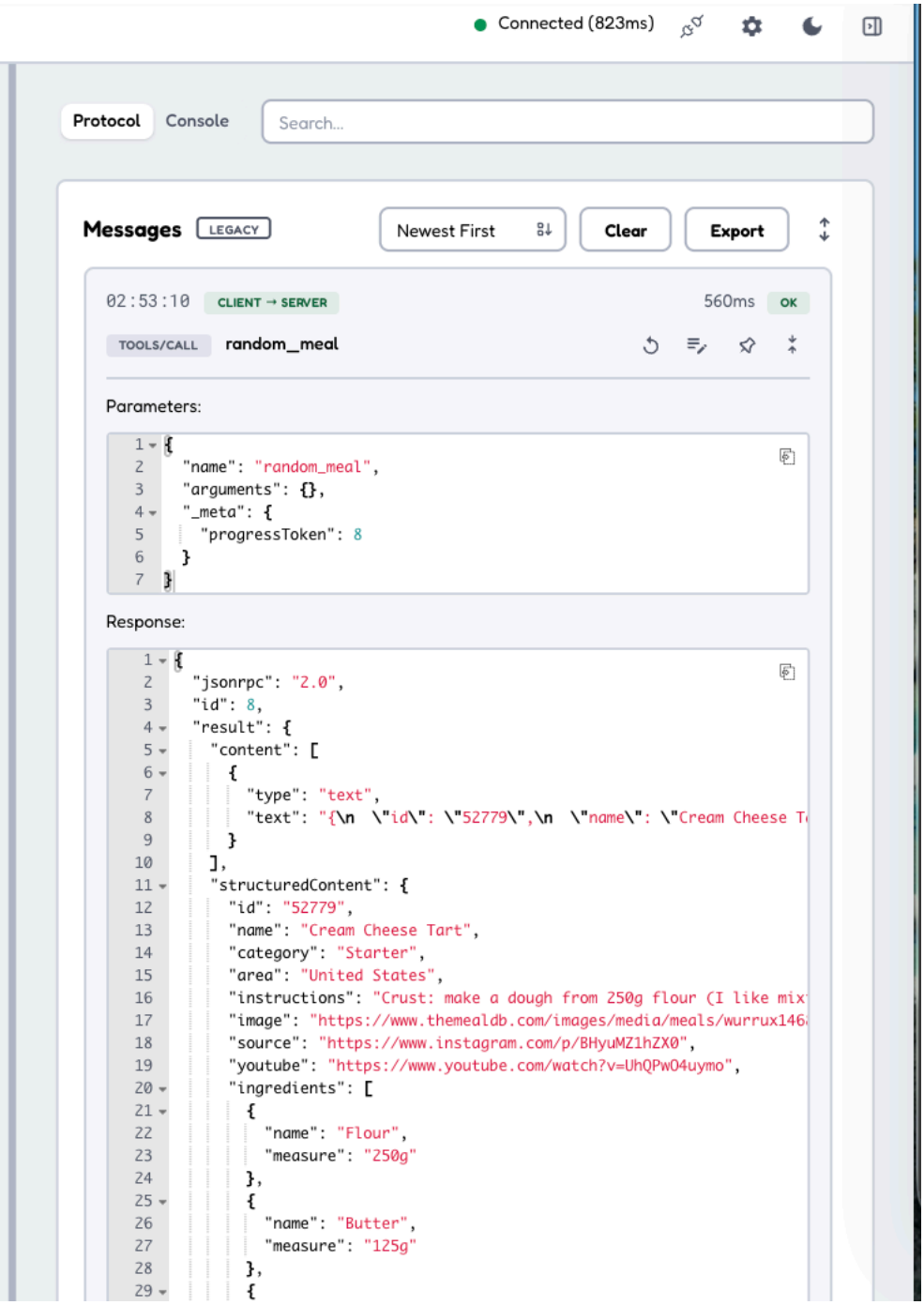


Figure 2.4. MCP Inspector: random_meal

2B. Domain server (s9275_recalls)

code/domain_server.py exposes exactly three tools over the s9275_rel data: search_notices (search), get_notice (detail lookup) and recall_summary (aggregate). The tools themselves live in code/domain_tools.py with no MCP import, so the same functions are reused by execute_tool in Part 4. Every result is one envelope, {ok, data, error}, defined once. Inputs are checked by one Pydantic model per tool; a bad call is returned as ok: false, not raised. The MCP parameters carry descriptions but no

limits, so an invalid call reaches call_tool and comes back in the envelope instead of in the framework's own error format. Logs go to stderr.

code/domain_tools.py — the envelope and the single way into the tools

```
def ok(data: Any) -> dict:
    return {"ok": True, "data": data, "error": None}


def fail(error: str) -> dict:
    return {"ok": False, "data": None, "error": error}

def call_tool(name: str, inputs: Any, repo) -> dict:
    params, error = check_inputs(name, inputs)
    if error:
        log.info("rejected %s %s: %s", name, inputs, error)
        return fail(error)
    try:
        return TOOLS[name](repo, params)
    except Exception as exc:
        reason = str(exc).splitlines()[0][:200] if str(exc) else ""
        log.error("%s failed: %s: %s", name, type(exc).__name__, reason)
        log.debug("traceback for the failure above", exc_info=True)
        return fail(f"{name} could not be completed: {type(exc).__name__}: {reason}")
```

code/domain_server.py — a tool is a thin wrapper

```
@mcp.tool()
def get_notice(
    recall_number: Annotated[str, Field(description="Recall number, format F-NNNN-YYYY,
e.g. F-0001-2025")],
) -> dict[str, Any]:
    """Full details of one recall notice and its firm, looked up by recall number.
    Returns {ok, data, error}; data = the notice."""
    return call_tool("get_notice", {"recall_number": recall_number}, repo())
```

Tool	Successful call	Intentionally invalid call	Returned error
search_notices	{"query": "peanut butter", "limit": 2}	{"query": "peanut butter", "limit": 500}	limit: Input should be less than or equal to 20
get_notice	{"recall_number": "F-0001-2025"}	{"recall_number": "ABC-12"}	recall_number: String should match pattern '^F-\d{4}-\d{4}$'
recall_summary	{"group_by": "category", "year": 2025}	{"group_by": "color"}	group_by: Input should be 'category', 'state', 'year' or 'firm'

The same calls were also recorded through the Inspector command-line mode in reports/hw05/raw/mcp_inspector_domain.json (make capture-domain).

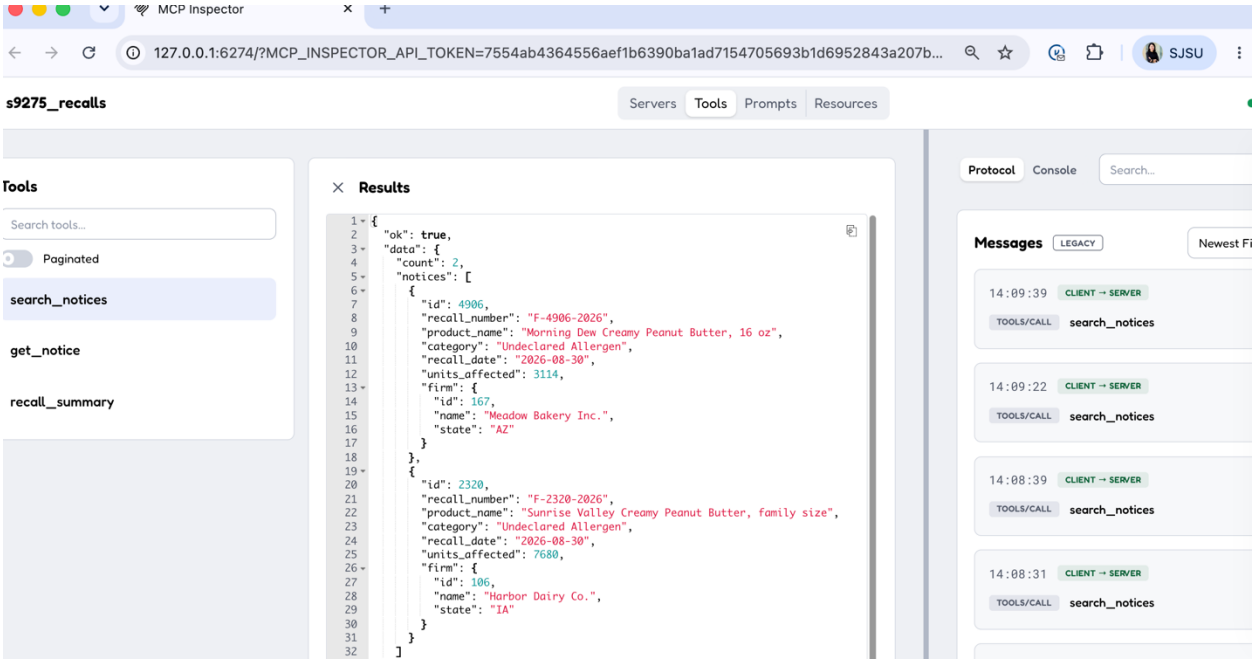


Figure 2.5. MCP Inspector: search_notices, successful call

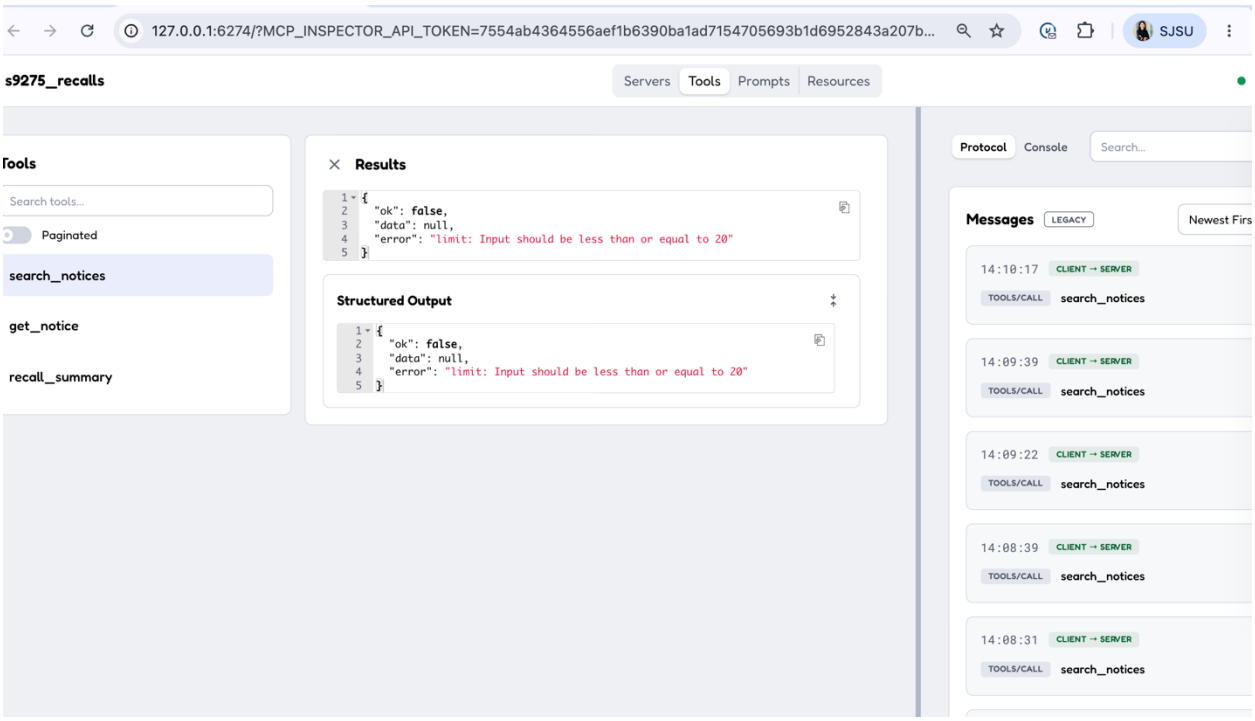


Figure 2.6. MCP Inspector: search_notices, invalid call

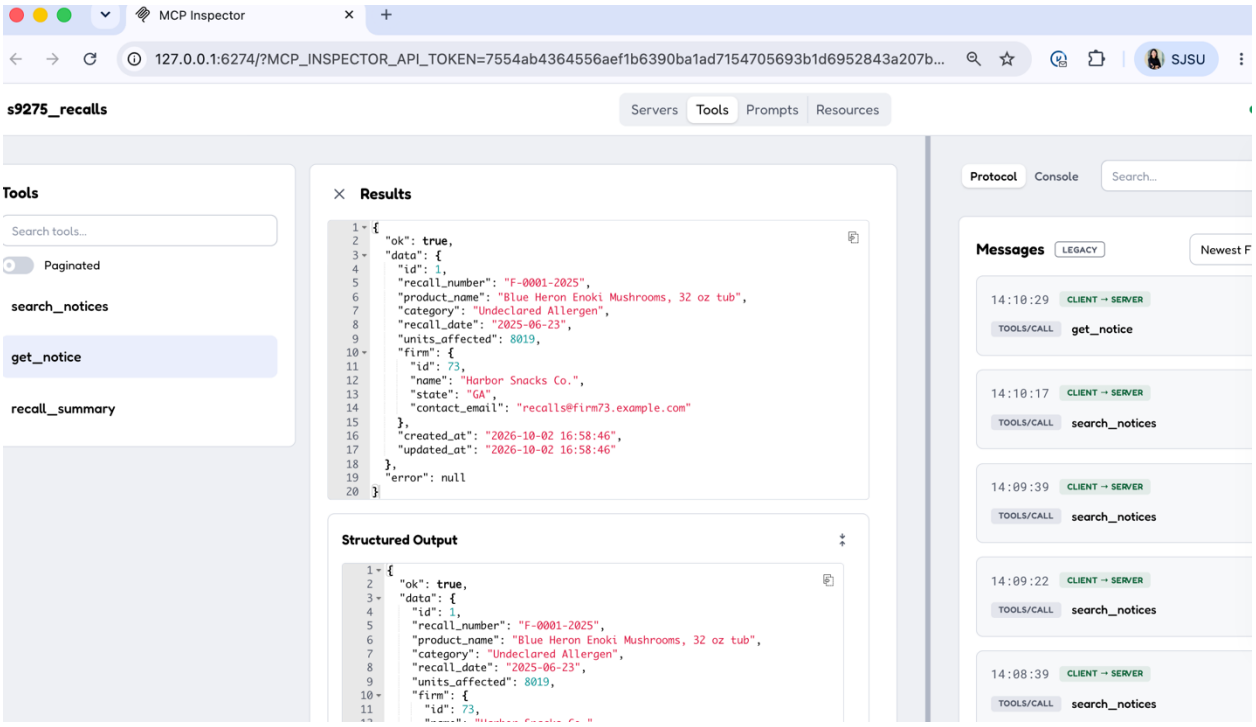


Figure 2.7. MCP Inspector: get_notice, successful call

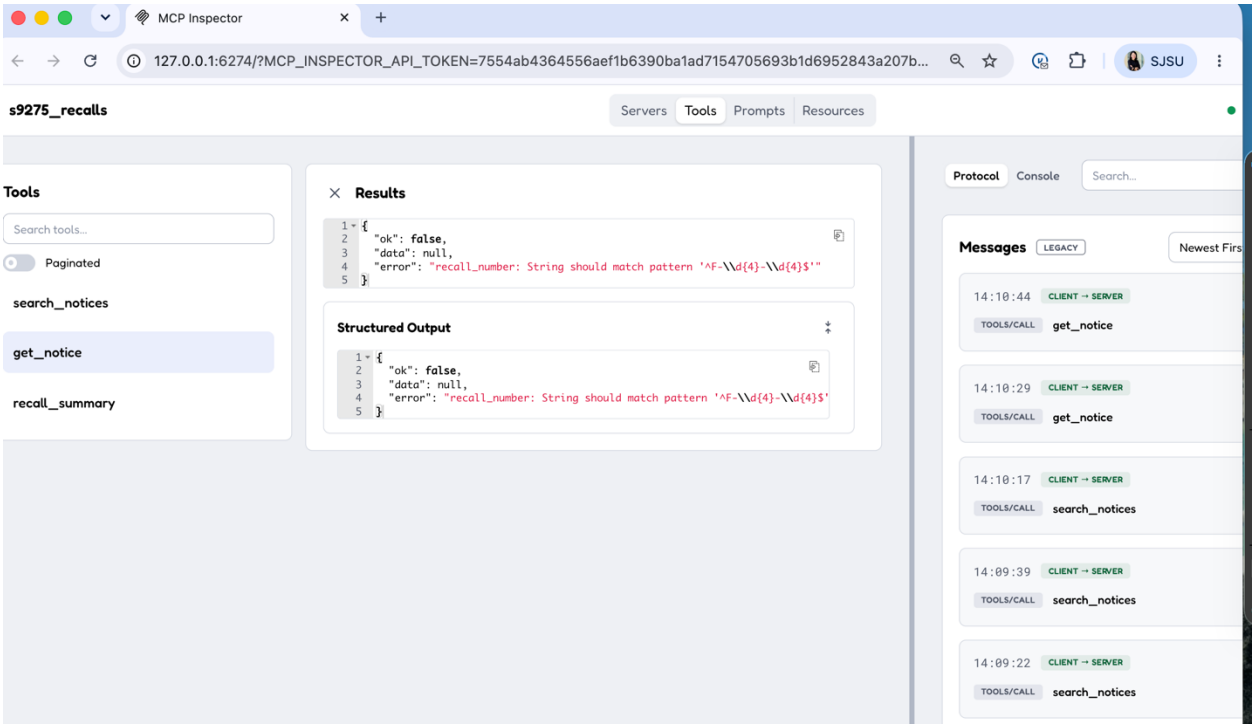


Figure 2.8. MCP Inspector: get_notice, invalid call

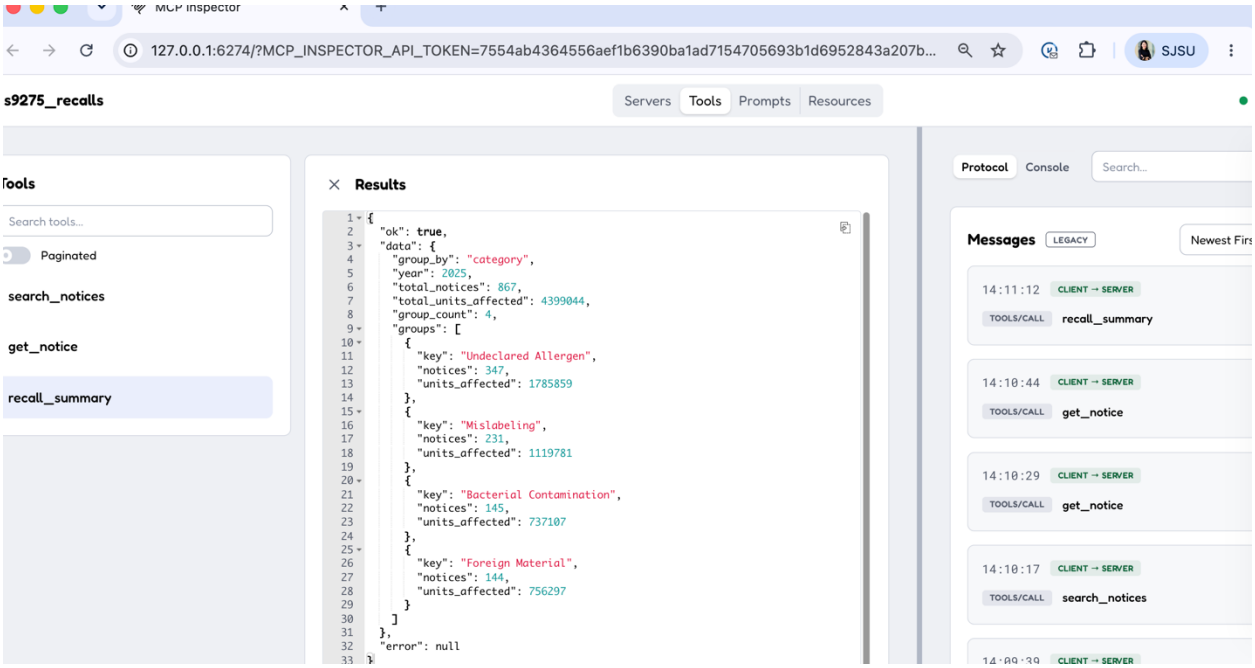


Figure 2.9. MCP Inspector: recall_summary, successful call

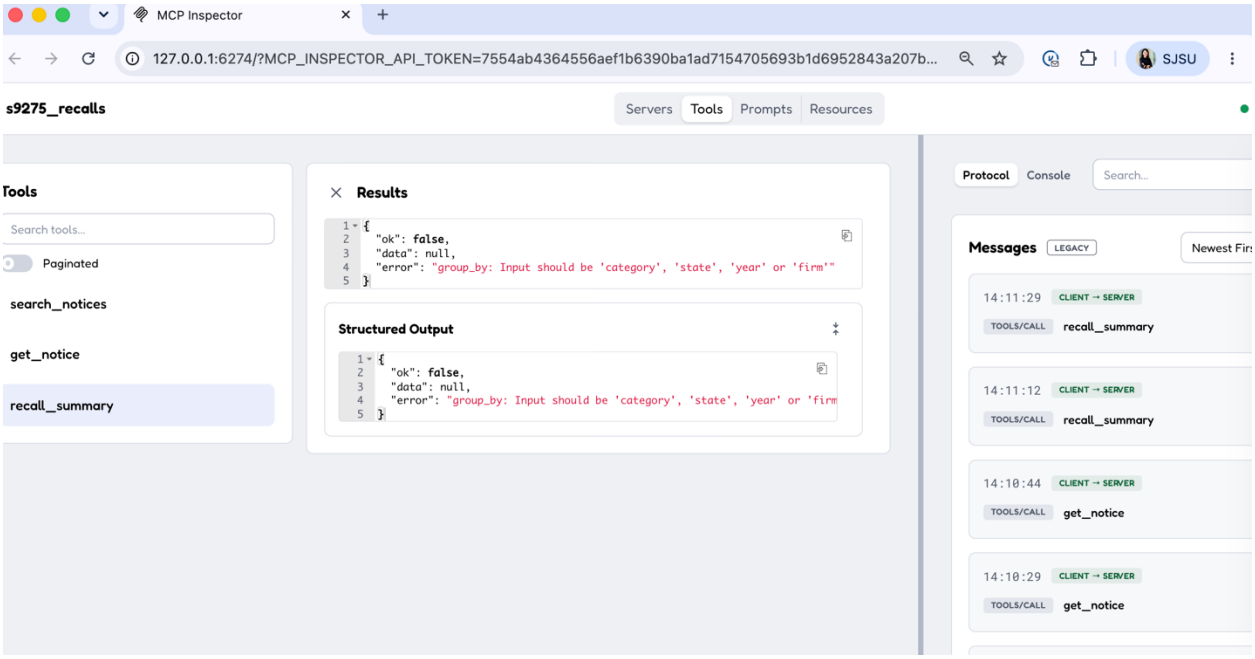


Figure 2.10. MCP Inspector: recall_summary, invalid call

Part 3 — Tool contracts under stress

3.1 Contracts and the rejected calls (item 18)

The schemas are generated from the Pydantic models (full text with descriptions: reports/hw05/TOOL_CONTRACTS.md). The rejected inputs and outputs are the Part 2B calls. Every check runs before any SQL, so none of these calls reached the database.

search_notices

Expected JSON input schema

```
{
  "type": "object", "additionalProperties": false, "required": ["query"],
  "properties": {
    "query": {"maxLength": 100, "minLength": 2, "type": "string"},
    "category": {"anyOf": [{"enum": ["Undeclared Allergen", "Bacterial Contamination",
"Foreign Material", "Mislabeling"], "type": "string"}, {"type": "null"}], "default":
null},
    "limit": {"default": 5, "maximum": 20, "minimum": 1, "type": "integer"}
  }
}
```

Rejected JSON input

```
{"query": "peanut butter", "limit": 500}
```

Returned error output

```
{"ok": false, "data": null, "error": "limit: Input should be less than or equal to 20"}
```

Why it was rejected: limit is an integer from 1 to 20 and the call sent 500. The cap keeps one call from pulling a large part of the 5,000-row table into a model's context.

get_notice

Expected JSON input schema

```
{
  "type": "object", "additionalProperties": false, "required": ["recall_number"],
  "properties": {
    "recall_number": {"pattern": "^F-\\d{4}-\\d{4}$", "type": "string"}
  }
}
```

Rejected JSON input

```
{"recall_number": "ABC-12"}
```

Returned error output

```
{"ok": false, "data": null, "error": "recall_number: String should match pattern '^F-
\\d{4}-\\d{4}$'"}
```

Why it was rejected: recall_number must look like F-0001-2025 (F, four digits, a four-digit year). ABC-12 cannot be a recall number, so the caller is told the format instead of getting "not found" after a lookup.

recall_summary

Expected JSON input schema

```
{
  "type": "object", "additionalProperties": false, "required": ["group_by"],
  "properties": {
    "group_by": {"enum": ["category", "state", "year", "firm"], "type": "string"},
    "year": {"anyOf": [{"maximum": 2026, "minimum": 2021, "type": "integer"}, {"type":
"null"}], "default": null}
  }
}
```

Rejected JSON input

```
{"group_by": "color"}
```

Returned error output

```
{"ok": false, "data": null, "error": "group_by: Input should be 'category', 'state',
'year' or 'firm'"}
```

Why it was rejected: group_by is an enum of four values and color is not one of them (there is no such column). Because only these values are accepted, free text never reaches the SQL.

3.2 Timeout and bounded exponential backoff (item 19)

code/retry_policy.py wraps every storage operation of the domain tools (RetryingRepo around MySQLRepo). Each attempt has a 2 s timeout; a failed attempt is retried after 0.1 s, then 0.2 s (doubling, capped at 1 s), three attempts in total. Only failures a retry can fix are retried (lost connection, timeout); invalid input and "not found" are not. When the attempts run out, call_tool turns the exception into an ok: false envelope.

code/retry_policy.py — the policy and the retry loop

```
@dataclass(frozen=True)
class RetryPolicy:
    max_attempts: int = 3          # the first try plus two retries
    timeout_s: float = 2.0         # one attempt may take this long
    base_delay_s: float = 0.1      # wait before the first retry
    multiplier: float = 2.0        # each later wait is this many times longer
    max_delay_s: float = 1.0       # ... but never longer than this

    def delay_before(self, attempt: int) -> float:
        return min(self.base_delay_s * self.multiplier ** (attempt - 2), self.max_delay_s)

def run_with_retry(operation: Callable[[], Any], policy: RetryPolicy = DEFAULT_POLICY,
                   sleep: Callable[[float], None] = time.sleep, name: str = "storage",
                   attempts: Optional[list] = None) -> Any:
    attempts = attempts if attempts is not None else []
    for attempt in range(1, policy.max_attempts + 1):
        waited = 0.0
        if attempt > 1:
            waited = policy.delay_before(attempt)
            sleep(waited)                                  # exponential backoff
        started = time.perf_counter()
        try:
            result = run_with_timeout(operation, policy.timeout_s)
        except Exception as exc:
            reason = (str(exc).splitlines() or [""])[0][:200]      # database errors run
to several lines
            attempts.append({"attempt": attempt, "waited_s": waited, "outcome": "failed",
                             "error": f"{type(exc).__name__}: {reason}",
                             "ms": round((time.perf_counter() - started) * 1000, 3)})
            if not is_transient(exc):
                raise                                      # retrying cannot help
            log.warning("%s attempt %d/%d failed: %s", name, attempt, policy.max_attempts,
reason)
            continue
        attempts.append({"attempt": attempt, "waited_s": waited, "outcome": "ok", "error":
None,
                         "ms": round((time.perf_counter() - started) * 1000, 3)})
        return result
    raise RetriesExhausted(f"{name} failed {policy.max_attempts} times; "
                           f"last error: {attempts[-1]['error']}")
```

Case (make retry-demo)	Attempts	Waits	Result	Total
1. Success on the first attempt	1	—	ok: true	1 ms
2. First attempt fails, retry succeeds	2	0.1 s	ok: true	105 ms
3. Every allowed attempt fails	3	0.1 s, 0.2 s	ok: false, clean error, no crash	311 ms
4. Storage slower than the timeout (0.2 s here)	3	0.1 s, 0.2 s	ok: false, StorageTimeout	916 ms

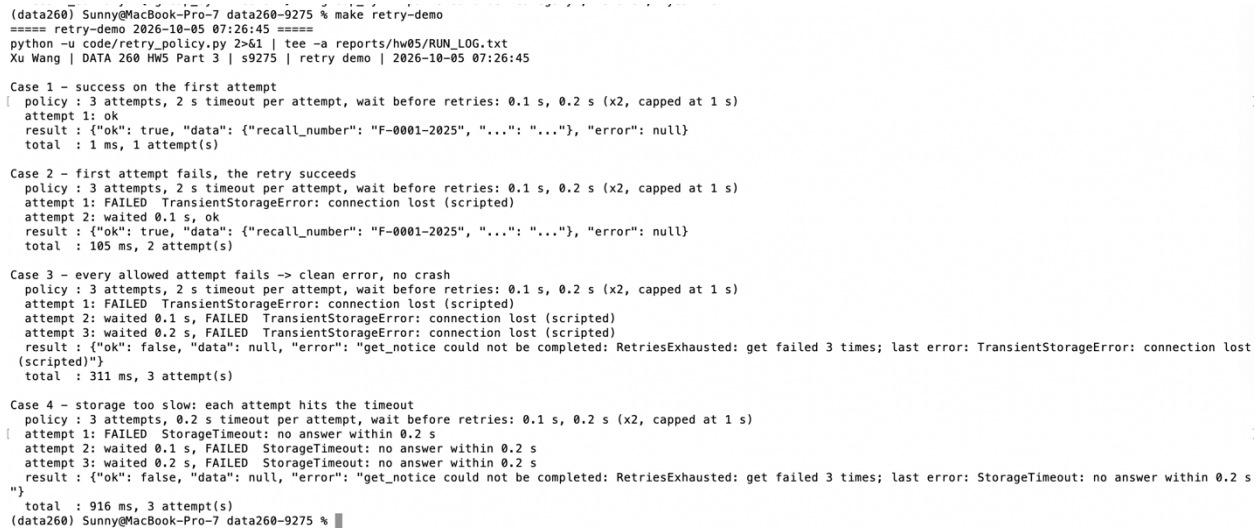


Figure 3.1. Terminal: make retry-demo: first-attempt success, fail then succeed, fail after all retries

3.3 Fault injection with VERIFY_SEED (item 20)

code/fault_injection.py puts a FlakyRepo between the retry policy and MySQL. Before every storage attempt it draws one number from random.Random(269275); below the failure rate the attempt fails. The same 50 valid calls (the three tools in turn) run at each rate, and the generator is re-seeded for each rate.

code/fault_injection.py — where the failures come from

```
class FlakyRepo:

    def __init__(self, inner, failure_rate: float, rng: random.Random):
        self.inner, self.failure_rate, self.rng = inner, failure_rate, rng
        self.injected = 0

    def _maybe_fail(self):
        if self.rng.random() < self.failure_rate:
            self.injected += 1
            raise TransientStorageError("injected failure")


def run_rate(rate: float, mysql: MySQLRepo) -> list[dict]:
    flaky = FlakyRepo(mysql, rate, random.Random(config.VERIFY_SEED))
    repo = RetryingRepo(flaky, DEFAULT_POLICY)
```

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100.0% (50/50)	2.41	13.99

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
20%	100.0% (50/50)	27.91	326.33
50%	88.0% (44/50)	105.72	329.20

Run of 2026-10-05 07:30 on MySQL s9275_rel. p99 is the nearest-rank percentile; with 50 values it is the slowest call. Raw data for all 150 calls: reports/hw05/raw/fault_injection_calls.csv.

Reproducibility: per call, 1/2/3 is the attempt that succeeded and X means all three attempts failed.

```
  0%  11111111111111111111111111111111111111111111111111
 20%  11111111112111111111121322111111111311111111121111
 50%  22111122211212313X21X1XX1111121311132321X1X1123111
```

These sequences equal what the seed alone predicts (expected_attempts() computes them without a database), and make verify-hw05 replays the 150 calls and compares them with the saved file. Latency changes between runs; the success/failure sequence does not.

```
(data260) Sunny@MacBook-Pro-7 data260-9275 % make fault-injection
===== fault-injection 2026-10-05 07:30:09 =====
python -u code/fault_injection.py 2>&1 | tee -a reports/hw05/RUN_LOG.txt
Xu Wang | DATA 260 HW5 Part 3 | s9275 | fault injection | 2026-10-05 07:30:09
VERIFY_SEED 269275 | 50 calls per rate | policy: 3 attempts, 2 s timeout per attempt, wait before retries: 0.1 s, 0.2 s (x2, capped at 1 s)

per call: 1/2/3 = succeeded on that attempt, X = failed after every allowed attempt
  0%  11111111111111111111111111111111111111111111111111  = seed prediction
 20%  11111111112111111111121322111111111311111111121111  = seed prediction
 50%  22111122211212313X21X1XX1111121311132321X1X1123111  = seed prediction

Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) | attempts | injected
                   0% |       100.0 % |              2.41 |            13.99 |       50 |        0
                  20% |       100.0 % |             27.91 |           326.33 |       59 |        9
                  50% |        88.0 % |            105.72 |           329.20 |       86 |       42

sequence signature (same seed -> same value): 6c4b498b7aab90a0
wrote reports/hw05/raw/fault_injection_calls.csv (150 rows)
wrote reports/hw05/raw/fault_injection_summary.json
```

Figure 3.2. Terminal: make fault-injection: 150 calls at 0%, 20%, 50%

3.4 Is the policy suitable? (item 21)

Interactive assistant: yes, with one change

- The added wait is bounded and short: at most 0.1 + 0.2 = 0.3 s. The slowest call in the experiment took 329 ms, while one turn of the local model takes seconds.
- Moderate trouble is hidden completely: at 20% all 50 calls succeeded (9 failures absorbed by 9 extra attempts); the mean rose from 2.4 ms to 27.9 ms.
- Heavy trouble fails fast and cleanly: at 50%, 6 of 50 calls failed (three independent attempts predict $0.5^3 = 12.5\%$), each returning ok: false in about 0.3 s, so the assistant can say the data is unavailable.
- What I would change is the timeout. Injected failures fail instantly, so the experiment does not show a call that hangs: the worst case is 3 × 2 s + 0.3 s = 6.3 s. Normal queries take 1–14 ms, so 1 s (worst case 3.3 s) fits a chat turn better. I would not add attempts: 5 attempts would add up to 1.5 s of waiting.

Batch processing: finish every item instead of answering fast

Setting	Interactive (tested)	Batch	Reason
Attempts	3	6–8	at a 50% failure rate, 6 attempts succeed 98.4% of the time ($1 - 0.5^6$)
Timeout per attempt	2 s (1 s preferred)	30 s	a heavy aggregate may legitimately take seconds
First wait / growth	0.1 s, ×2	1 s, ×2	a restarting database needs seconds, not 0.3 s

Setting	Interactive (tested)	Batch	Reason
Cap on one wait	1 s	30–60 s	waits of 1, 2, 4, 8, 16, 30 s
Jitter	none	±50% random	workers that fail together should not retry together
When retries run out	return ok: false	record the item, re-run it later	the tools only read, so repeating is safe

Part 4 — One safe tool entry point and offline tests

4.1 execute_tool (item 22)

execute_tool(name, inputs) in code/tool_entry.py is the only function the agent uses to reach a tool. It accepts a dict or JSON text, applies the Part 5 safety rule, calls call_tool, and always returns the {ok, data, error} envelope as a JSON string. No exception leaves it.

code/tool_entry.py

```
def execute_tool(name: str, inputs: Any, repo: Optional[Any] = None) -> str:
    try:
        if isinstance(inputs, str):                     # arguments arrived as text
            try:
                inputs = json.loads(inputs)
            except ValueError:
                return json.dumps(fail("inputs is not valid JSON"))
        blocked = safety_check(name, inputs)            # Part 5: policy first, before
any work
        if blocked:
            log.warning("%s %s -> %s", name, inputs, blocked)
            return json.dumps(fail(blocked))
        envelope = call_tool(name, inputs, repo if repo is not None else default_repo())
    except Exception as exc:                            # last line of defence; call_tool
        log.error("execute_tool(%r) failed: %r", name, exc)   # already catches tool
errors
        envelope = fail(f"{name} could not be completed: {type(exc).__name__}")
    # default=str: a date or Decimal that slipped into data becomes text instead of an
error
    return json.dumps(envelope, default=str)
```

4.2 Offline test runner (items 23–24)

code/test_tools.py uses plain assert statements and prints PASS or FAIL per test and an X/Y summary (make test-tools). The tests run offline through dependency injection: execute_tool takes a repo argument, and the tests pass an InMemoryRepo (3 firms, 6 notices in a Python list) instead of MySQL. The repo records which methods were called, so a test can show that a rejected call never reached storage. The agent tests use MockModel instead of Ollama, and the last test asserts that no database driver or model client was imported.

code/test_tools.py — the contract check used by every test, and one of the invalid-input tests

```
def run(name, inputs, repo):
    text = execute_tool(name, inputs, repo=repo)
    assert isinstance(text, str), f"execute_tool returned {type(text).__name__}, not str"
    envelope = json.loads(text)
    assert set(envelope) == {"ok", "data", "error"}, f"unexpected keys {sorted(envelope)}"
    if envelope["ok"]:
```

```
        assert envelope["error"] is None, "error must be null on success"
    else:
        assert envelope["data"] is None and envelope["error"], "a failure needs data null
and an error text"
    return envelope

def test_search_rejects_limit_500():
    """search_notices: limit 500 is rejected (Part 3 case 1) and storage is not touched"""
    repo = InMemoryRepo()
    result = run("search_notices", {"query": "peanut butter", "limit": 500}, repo)
    assert result["ok"] is False
    assert "limit" in result["error"] and "20" in result["error"], result["error"]
    assert repo.calls == [], f"storage was called: {repo.calls}"
```

Tests	What they cover
01–05	valid inputs for each tool: search (2), get_notice (1), recall_summary (2)
06–08	the three invalid inputs from Part 3: limit 500, recall_number ABC-12, group_by color
09–12	not found, unknown tool or argument, inputs as JSON text, storage that raises
13–14	retry policy: recover on attempt 2; give up cleanly after 3
15–18	Part 5: safety rule; agent stops at max_steps (MockModel), completes, stops on a safety block
19	the run was offline

```
(data260) Sunny@MacBook-Pro-7 data260-9275 % make test-tools
===== test-tools 2026-10-05 11:15:39 =====
python -u code/test_tools.py 2>&1 | tee -a reports/hw05/RUN_LOG.txt
Xu Wang | DATA 260 HW5 Parts 4-5 | s9275 | offline tests | 2026-10-05 11:15:40

PASS  01 search_notices: valid query finds both hummus notices, newest first
PASS  02 search_notices: category filter and limit are applied
PASS  03 get_notice: valid recall number returns that notice and its firm
PASS  04 recall_summary: by category, counts and unit totals add up
PASS  05 recall_summary: by state for one year only counts that year
PASS  06 search_notices: limit 500 is rejected (Part 3 case 1) and storage is not touched
PASS  07 get_notice: recall_number 'ABC-12' is rejected (Part 3 case 2) before any lookup
PASS  08 recall_summary: group_by 'color' is rejected (Part 3 case 3)
PASS  09 get_notice: well-formed but unknown number -> ok false, 'not found'
PASS  10 execute_tool: unknown tool, unknown argument, missing argument -> ok false
PASS  11 execute_tool: inputs may arrive as JSON text; broken JSON -> ok false
PASS  12 execute_tool: storage that raises -> ok false, no exception escapes
PASS  13 retry policy: one transient failure, then success on attempt 2 after a 0.1 s wait
PASS  14 retry policy: three failures -> clean ok false, waits were 0.1 s then 0.2 s
PASS  15 safety rule: recall_summary by firm is blocked, storage untouched; by category still works
PASS  16 agent loop: a model that keeps asking for tools is stopped at max_steps
PASS  17 agent loop: search, then detail, then a text answer -> completed in 3 steps, all logged
PASS  18 agent loop: a blocked tool call ends the run at once with stop reason safety_block
PASS  19 offline: no database driver, ORM or model client was imported by this run

19/19 tests passed
(data260) Sunny@MacBook-Pro-7 data260-9275 %
```

Figure 4.1. Terminal: make test-tools: PASS/FAIL for each test and the 19/19 summary

Part 5 — Agent loop, safety rule and reflection

5.1 Safety rule

Rule: the assistant must not rank named firms by recalls. recall_summary with group_by = firm returns the ten companies with the most notices; published by an assistant from data that may be incomplete, that is a "worst offenders" list of business names. Totals by category, state or year name no company and stay allowed. This is a policy decision, not an input check: {"group_by": "firm"} is valid input for the tool. The rule runs inside execute_tool before any query (see 4.1) and returns the ok: false envelope without raising.

code/tool_entry.py

```
def safety_check(name: str, inputs: Any) -> Optional[str]:
    if name == "recall_summary" and isinstance(inputs, dict):
        if str(inputs.get("group_by", "")).strip().lower() == "firm":
            return (f"{SAFETY_PREFIX}: recall_summary may not group by firm, because the "
                    f"assistant must not rank named companies by recalls. "
                    f"Group by category, state or year instead.")
    return None
```

```
(data260) Sunny@MacBook-Pro-7 data260-9275 % make safety-demo
===== safety-demo 2026-10-05 07:59:19 =====
python -u code/tool_entry.py --safety-demo 2>&1 | tee -a reports/hw05/RUN_LOG.txt
Xu Wang | DATA 260 HW5 Part 5 | s9275 | safety rule demo | 2026-10-05 07:59:19
Rule: the assistant must not rank named firms by recalls (recall_summary with group_by = firm).

ALLOWED  execute_tool("recall_summary", {"group_by": "category", "year": 2025})
  ok    : true
  data  : total_notices 867; Undeclared Allergen: 347, Mislabeling: 231, Bacterial Contamination: 145, Foreign Material: 144
  error : null

BLOCKED  execute_tool("recall_summary", {"group_by": "firm", "year": 2025})
  returned: {"ok": false, "data": null, "error": "blocked by safety rule: recall_summary may not group by firm, because the assistant must not rank named companies by recal
ls. Group by category, state or year instead."}
```

Figure 5.1. Terminal: make safety-demo: one allowed call and one blocked call

5.2 Agent loop

run_agent(user_input) in code/agent_loop.py sends the question and the three tool schemas to qwen3:8b through Ollama. One step is one model call plus every tool it asked for. The for loop is the turn counter and max_steps (default 5) its limit. Each run is appended to reports/hw05/raw/agent_runs.jsonl as one JSON line: run_id, model, max_steps, every step with its tool calls (tool, inputs, result), stop_reason, step_count, tool_call_count, final_answer and duration.

Stop reason	When
completed	the model answered in text and asked for no tool
safety_block	execute_tool refused a call under the safety rule; the loop ends at once
max_steps	the step budget ran out before a final answer
model_error	the model call itself failed

code/agent_loop.py — the loop inside run_agent

```
for step in range(1, max_steps + 1):                        # the turn counter
    turn_started = time.perf_counter()
    try:
        reply = model.complete(messages, tools=tools)
    except Exception as exc:                                # Ollama not running, model
missing...
        log.error("model call failed: %r", exc)
        stop_reason = "model_error"
```

```
        final_answer = f"The model could not be reached: {type(exc).__name__}: {exc}"
        break
    record = {"step": step,
              "model_ms": round((time.perf_counter() - turn_started) * 1000),
              "input_tokens": getattr(reply, "input_tokens", None),     # None for
MockModel
              "output_tokens": getattr(reply, "output_tokens", None),
              "assistant_text": reply.text,
              "tool_calls": []}
    run["steps"].append(record)

    if not reply.tool_calls:                              # a plain answer ends the run
        stop_reason, final_answer = "completed", strip_thinking(reply.text)
        break

    messages.append({"role": "assistant", "content": reply.text,
                     "tool_calls": [{"function": {"name": c["name"], "arguments":
c["arguments"]}}
                                    for c in reply.tool_calls]})
    blocked = None
    for call in reply.tool_calls:
        result_text = execute_tool(call["name"], call["arguments"], repo=repo)
        envelope = json.loads(result_text)
        record["tool_calls"].append({"tool": call["name"], "inputs": call["arguments"],
                                     "result": envelope})
        messages.append({"role": "tool", "tool_name": call["name"], "content":
result_text})
        if is_safety_block(envelope):
            blocked = envelope["error"]
            break
    if blocked:                                           # policy refusal: stop at once
        stop_reason = "safety_block"
        final_answer = f"I can't help with that request. {blocked}"
        break
else:
    final_answer = (f"Stopped after {max_steps} steps without a final answer "
                    f"(max_steps limit).")
```

```
(data260) Sunny@MacBook-Pro-7 data260-9275 % python code/agent_loop.py "How many recall notices were there in 2025, broken down by category?"
Xu Wang | DATA 260 HW5 Part 5 | s9275 | agent run | model qwen3:8b | 2026-10-05 08:11:39

Q: How many recall notices were there in 2025, broken down by category?
  step 1 (45.2 s, 556 tokens in / 30 out): recall_summary({"group_by": "category", "year": 2025}) -> ok
  step 2 (75.0 s, 785 tokens in / 121 out): model answered in text
  stop reason: completed | steps: 2/5 | tool calls: 1 | 121.3 s
A: In 2025, there were a total of 867 recall notices broken down by category as follows:

- **Undeclared Allergen**: 347 notices affecting 1,785,859 units.
- **Mislabeling**: 231 notices affecting 1,119,781 units.
- **Bacterial Contamination**: 145 notices affecting 737,107 units.
- **Foreign Material**: 144 notices affecting 756,297 units.
```

Figure 5.2. Terminal: one agent run with qwen3:8b (question, steps, tool calls, stop reason, answer)

5.3 Additional offline tests

Two tests were added to the Part 4 suite; both use the in-memory repo, and the second uses MockModel, so neither needs a model, an API or a database. They are lines 15 and 16 in Figure 4.1.

code/test_tools.py

```
def test_safety_rule_blocks_firm_ranking():
    """safety rule: recall_summary by firm is blocked, storage untouched; by category
still works"""
    repo = InMemoryRepo()
```

```
    blocked = run("recall_summary", {"group_by": "firm"}, repo)
    assert blocked["ok"] is False and blocked["data"] is None
    assert blocked["error"].startswith(SAFETY_PREFIX), blocked["error"]
    assert repo.calls == [], f"a blocked call reached storage: {repo.calls}"
    as_text = run("recall_summary", '{"group_by": "FIRM", "year": 2025}', repo)   # same
call, other spelling
    assert as_text["error"].startswith(SAFETY_PREFIX) and repo.calls == []
    allowed = run("recall_summary", {"group_by": "category"}, repo)
    assert allowed["ok"] is True and repo.calls == ["summary"]

def test_agent_stops_at_max_steps():
    """agent loop: a model that keeps asking for tools is stopped at max_steps"""
    run, model, repo, logged = agent_run([wants("search_notices", query="hummus")],
max_steps=3)
    assert run["stop_reason"] == "max_steps", run["stop_reason"]
    assert run["step_count"] == 3 and model.calls == 3, (run["step_count"], model.calls)
    assert run["tool_call_count"] == 3 and repo.calls == ["search"] * 3
    assert len(logged) == 1 and logged[0]["stop_reason"] == "max_steps"
    assert [len(step["tool_calls"]) for step in logged[0]["steps"]] == [1, 1, 1]
```

5.4 Metrics

Five scenarios with the local model (make agent-scenarios, run of 2026-10-05 08:30, qwen3:8b, temperature 0, tools reading MySQL). Each row can be matched to its line in agent_runs.jsonl by run_id.

	Scenario	Max steps	Steps	Tool calls	Stop reason	Tools called	run_id
S1	one aggregate call, then an answer	5	2	1	completed	recall_summary	75f5e739
S2	search, then detail lookup	5	3	2	completed	search_notices, get_notice	52422676
S3	a request the safety rule refuses	5	1	1	safety_block	recall_summary	d680adb7
S4	the S2 question with a budget of 2	2	2	2	max_steps	search_notices, get_notice	5ffba3f4
S5	an invalid tool input	5	2	1	completed	get_notice	228f75ce

- S2: search_notices found F-4906-2026, get_notice returned the firm and its email; the answer matches the tool result. S4 made the same two calls but had no step left to answer: a question that needs N tool calls needs N + 1 steps.
- S5: get_notice("4906") failed input validation. That is not a safety block, so the error went back to the model, which explained the required format. completed means the loop ended normally, not that the question was answered.
- An earlier batch (raw/agent_scenarios_run1.json) ended S2 and S4 with safety_block: after the search the model called recall_summary by firm to find the firm's email. I changed only the tool descriptions and the system prompt (get_notice returns the firm and its email; recall_summary is for statistics only), not the rule or the loop. This is one run per scenario per batch, so it shows the change fixed this question, not that the model always picks the right tool.

- Model steps took 21–129 s in this batch. The 6.0 GB model does not fit fully on the GPU of this 8 GB machine (ollama ps: 24% CPU / 76% GPU), so seconds are not comparable between runs; steps, tool calls and stop reasons are.

5.5 Reflection

Run 3adaf676 in raw/agent_runs.jsonl (qwen3:8b, max_steps 5)

A harmless question: "Find the most recent peanut butter recall and tell me which firm issued it and that firm's contact email."

Step 1. The harness sent the system prompt, the question and the three tool schemas to the model. The model asked for search_notices(query="peanut butter", limit=1). The harness passed the call to execute_tool. The safety check found nothing, the query ran, and the envelope came back ok: true with notice F-4906-2026. The harness added the result to the conversation.

Step 2. The model should have called get_notice, which returns the firm's email. It called recall_summary(group_by="firm", year=2026) instead. My safety rule forbids grouping by firm, because the assistant must not rank named companies by recalls. execute_tool returned ok: false with a "blocked by safety rule" error and ran no query.

Why it stopped. A safety-rule block. The harness recognised the block, ended the loop at once and wrote the refusal itself, after 2 of 5 steps and 2 tool calls. The model was not called again.

What I learned. The rule did its job, but the system was fragile: a block is final, so one wrong tool choice cost the user a legitimate answer. I did not loosen the rule. I rewrote the tool descriptions and the system prompt to say that get_notice returns the firm and its email and that recall_summary is for statistics only. In the next batch the same question completed in 3 steps (run 52422676). A further option is to return a block to the model as an ordinary error, so it can try another tool within max_steps, as the loop already does for invalid inputs. A strict rule is not enough. The tool descriptions must also guide the model to the right tool.

AI use

1. What I used an AI assistant for, and what I did myself

AI assistant (Claude):

- Explained the assignment and put the steps in order.
- Review the design and give me advice and options to choose.
- Wrote the first version of the code for each part: the migration script, the firms/notices API, the Redux client, the two MCP servers, the retry policy, the fault-injection experiment, execute_tool, the offline tests and the agent loop. It tested these in its own environment before handing them over.
- Read the output and logs I pasted back and diagnosed problems.
- Review my reflection and put my reflection into English.

Me:

- First designed the whole structure and ask AI to modify my ideas
- Made these design decisions: contact_email and recall_number as the unique fields; a migration script instead of rebuilding the tables; the safety rule (no ranking of named firms by recalls).
- Reviewed and approved the retry-policy numbers and the design of the three domain tools that the assistant proposed.

- Ran all the code on my Mac (MySQL, conda env data260, Ollama qwen3:8b) and compared each output with what was expected.
- Tested every tool by hand in the MCP Inspector web UI and took the Inspector, Postman, database and React screenshots.
- Found several places where the result did not match what the assistant had told me (one is described below).
- Drafted the analysis text in METRICS.md and wrote my reflection in Chinese.

2. One AI-produced output that was wrong

For the invalid-input test in Part 2A, the assistant told me to type limit = 99 into the Inspector form for search_meals_by_name (allowed range 1-25) and said I would see a validation error. I did that and got no error. The tool ran and returned meals as if the input were valid.

3. How I detected the problem

I ran the call in the Inspector web UI and looked at the result panel: no error message, a normal list of meals. I reported this to the assistant with a screenshot. The cause was the Inspector (version 2.9), not my server. Its form reads maximum: 25 from the tool schema and silently changes 99 to 25 when the input box loses focus, so the value 99 was never sent to the server.

4. What I changed and why it works now

- In the web UI I now enter invalid values with "Edit as JSON", which sends the arguments exactly as typed.
- code/mcp_capture.py (make capture-meals) sends the same call through the Inspector command-line mode, which does not change the arguments.

It works because the raw value now reaches the server. raw/mcp_inspector_meals.json records the call with "limit": 99 and the server's answer: isError: true, "Input should be less than or equal to 25". What I took from this: a tool's input check is tested only if the invalid value is really sent, so I check what the client sent, not only what I typed.

Verification and how to run

make verify-hw05 (verify_hw05.py) is the smoke test. It starts the API on port 8275 and both MCP servers, creates and deletes one temporary firm and notice through the API, replays the 150 fault-injection calls, runs the offline tests and one live agent run, and writes reports/hw05/verification.json with the commit hash, configuration and the result of each check. Result for commit f03debd: 61 passed, 0 failed.

Command	What it does
make db-init seed migrate-hw05	build s9275_rel with the HW5 schema (once)
make run-hw05 / make run-frontend	API on http://localhost:8275, React client on http://localhost:5173
make inspect-meals / make inspect-domain	MCP Inspector on each server
make retry-demo / make fault-injection	Part 3 demonstrations and the 150-call experiment
make test-tools	offline tests (no MySQL, no model)
make safety-demo / make agent-scenarios	Part 5 safety rule and the five scenarios (needs Ollama)
make verify-hw05	self-check → verification.json